IPAddress

Java and Go libraries for handling IP addresses and subnets, both IPv4 and IPv6

by Sean C Foley

Contents

Contents

Benefits of this Library

Java versus Golang Feature Matrix

Code Examples

Supported IP Address Parsing Formats

Core Types

Parse String Representation of IP Address, Subnet or Host Name

Addresses from Numeric Values

Golang Address Keys

Golang Zero Values

Networks

Prefix Length Handling

Address Sections

IP Address Ranges

Address Tries

IP Address Collections

IP Address Operations

Parse String Representations of MAC Address

MAC Address Operations

IPv6 – MAC Address Integration

Address Framework

Conversion to String Representation of Address or Subnet

Containment and Membership

Sorting and Comparisons

DNS Resolution and URLs

Make your IPv4 App work with IPv6

Benefits of this Library

The IPAddress library was intended to satisfy the following primary goals:

- **Parsing of all host name and ipv4/ipv6 address formats in common usage** plus some additional formats (see below or see javadoc for the `IPAddressString` class or godoc for the struct `IPAddressString` for an extensive list)
- **Parsing and representation of subnets**, either those specified by network prefix length or those specified with ranges of segment values. For example, all strings in the list below represent the same IPv4 subnet:
  - with CIDR network prefix length: `1.2.0.0/16`
  - with mask: `1.2.0.0/255.255.0.0`
  - wildcard segments: `1.2.*.*`
  - range segments: `1.2.0-255.0-255`
  - range using inet_aton format: `0x1.0x2.0x0-0xffff`
  - with CIDR network prefix length using binary format: `0b1.0b10.0b0.0b0/16`
  - SQL-style single wildcards to end segments: `1.2.___.___`
  - IPv4 mapped IPv6: `::ffff:1.2.0.0/112`
  - Hexadecimal values: `0x01020000-0x0102ffff`
  - Octal values: `000100400000-000100577777`

- Binary values: `0b00000001000000100000000000000000-0b00000001000000101111111111111111`

- **Allow the separation of address parsing from host parsing**. In some cases you may have an address, in others you may have a host name, in some cases either one, so this supports all three options (for instance, when validating invalid input "1.2.3.a" as an address only, it will not be treated as a host with DNS lookup attempted in the way that Java's `InetAddress.getByName` does)

- **Allow control over which formats are allowed when parsing**, whether IPv4/6, or subnets, or inet_aton formats, and so on.

- **Produce all common address strings of different formats for a given IPv4 or IPv6 address and produce collections of such strings**
  Some addresses can have hundreds of thousands of potential string representations (when you consider hex capitalization, ipv6 compression, and leading zeros, the various IPv4 and IPv6 formats, and combinations of all the above), although there are generally a handful of commonly used formats. Generating these strings can help when searching or matching addresses in databases or text.

- **Support parsing of all common MAC Address** formats in usage and produce all common MAC address strings of different formats

- **Integration of MAC Address with IPv6** with standard conversions

- **Integration of IPv4 Address with IPv6** through common address conversions

- **Polymorphism** is a key goal. The library maintains an architecture and an address framework of interfaces that allow most library functionality to be independent of address type or version, whether IPv4, IPv6 or MAC. This allows for code which supports both IPv4 and IPv6 transparently. You can write generic non-version-specific code to validate addresses, connect to addresses, produce address strings, mask addresses, etc. You can make use of the address framework which is agnostic towards address version.

- **Thread-safety and immutability**. The core types (host names, address strings, addresses, address sections, address segments, address ranges) are all immutable. They do not change their underlying value. For concurrency in apps (Java threads, Kotlin coroutines, or Go goroutines) this is valuable.

- **Address modifications**, such as altering prefix lengths, masking, splitting into sections and segments, splitting into network and host sections, reconstituting from sections and segments

- **Address operations and subnetting**, such as obtaining the prefix block subnet for a prefixed address, iterating, spliterating or streaming through subnets, iterating, spliterating or streaming through prefix blocks, incrementing and decrementing addresses by integer values, reversing address bits for endianness or DNS lookup, set-subtracting subnets from other subnets, subnetting, intersections of subnets, merging subnets, checking containment of addresses in subnets, listing subnets covering a span of addresses

- **Sorting and comparison** of host names, addresses, address strings and subnets

- **Address data structures** including the trie, associative trie, corresponding sets and maps, providing additional address operations such as group containment operations, sorting operations, prefix block operations, and alternative subnet traversals. Also, the two collections options, the IP address sequential range list backed by an array of sequential ranges, and the IP address containment trie backed by a trie, both optimized and efficient types to maintain collections of addresses.

- **Integrate with standard types.** For Java, this includes the primitive signed types and the standard library classes `java.net.InetAddress`, `java.net.Inet6Address`, `java.net.Inet4Address`, and `java.math.BigInteger`. For Go, this includes the primitive unsigned types and the Go standard library types `net.IP`, `net.IPAddr`, `net.IPMask`, `net.IPNet`, `net.TCPAddr`, `net.UDPAddr`, `net/netip.Addr`, `net/netip.AddrPort`, `net/netip.Prefix`, and `big.Int`.

- **Making address manipulations easy**, so you do not have to worry about longs/ints/shorts/bytes/bits, signed/unsigned, sign extension, ipv4/v6, masking, iterating, and other implementation details.

Java Versus Golang Feature Matrix

The basic goals remain the same for both Java and Go libraries. This matrix is a summary of the feature differences.

Feature	Java	Golang
Core types (IP/MAC strings, IPv4/v6/MAC addresses, sequential ranges, host names, base types for the like)	✅	✅
Core types are immutable	✅	✅
Rich type system for addresses, address sections, address segments, address divisions, address division groupings, and address ranges	✅	✅
Comparison of instances of all such types	✅	✅
Subnet containment checks	✅	✅
Sequential range containment checks	✅	✅
Merging subnets	✅	✅
Merging sequential ranges	✅	✅
Spanning subnets with CIDR blocks	✅	✅
Spanning sequential ranges with CIDR blocks	✅	✅
Spanning subnets with sequential blocks	✅	✅

Feature	Java	Golang
Spanning sequential ranges with sequential blocks	✅	✅
Conversion to/from MAC from/to IPv6 Addresses	✅	✅
Conversion to/from IPv4 from/to IPv6 Addresses	✅	✅
Subnet iterators	✅	✅
Sequential range iterators	✅	✅
Subnet prefix iterators	✅	✅
Sequential range prefix iterators	✅	✅
Spliterator and stream iterator alternatives	✅	
Parsing many address and subnet formats	✅	✅
String generation of many address and subnet formats	✅	✅
Address string collections	✅	
Address increment/decrement/enumerate	✅	✅
Masking, reversing, subtracting, intersecting, joining operations	✅	✅
Prefix length operations	✅	✅
Framework of address interfaces for polymorphic code	✅	✅
Address tries	✅	✅
Associative address tries	✅	✅
Containment trie IP Address collections	✅	Future
Sequential range list IP address collections	✅	Future
Integration with standard library maps and collections	✅	✅
Parse IP strings directly to sequential ranges	✅	✅
UNC Host, DNS, & IPv6 Base 85 string parsing and generation	✅	✅
Parse IP strings directly to division groupings	✅	
Prefix Block Allocator	✅	✅
Serialization	✅	

Code Examples

This document provides in-depth and extensive documentation for the library, and includes some code snippets. However, for common use-cases, you may wish to go straight to the Java code examples or Go code examples which cover a wide breadth of common use-cases. The code examples are focused more on covering common use-cases and operations, while this document is focused more on covering all areas in more detail with smaller code snippets. This document focuses more on one area at a time, while the examples are useful in showing how to combine the functionality to achieve various end results.

Supported IP Address Parsing Formats

This includes, those supported by the well-known routines inet_aton and inet_pton, the subnet formats listed above, all combinations of the above, and others:

- all the formats supported by `inet_pton` and `inet_aton`
- all the formats supported by `nmap`
- all the formats produced by `netstat` involving hosts/addresses with ports/services
- the subnet formats listed above, whether prefixed, masked, wildcards, ranges
- IPv6 canonical, compressed `1::1`, mixed `1:2:3:4:5:6:1.2.3.4`, `[bracketed]:port`, `::1:service`, and so on
- Hex values
- Binary values
- IPv6 base 85 values
- `*` represents all addresses both ipv4 and ipv6
- `/x` where x is a positive integer, with no associated address, is interpreted as the network mask for prefix length x
- ″ ″ the empty string is considered the default loopback or the default zero address

For a more detailed list or formats parsed, some examples are below, or see the javadoc or godoc for `IPAddressString`.

Subnet formats

- **CIDR (Classless Inter-Domain Routing) prefix length subnets**
  Adding the prefix length `/x` creates the address or subnet for that network prefix length. It will be a subnet if the host for prefix length 'x' is entirely zero. In other words, he subnet 1.2.0.0/16 is the set of all addresses starting with the 16-bit prefix "1.2".

- **Wildcard `*` `_` and range `-` subnets:**
  `*` denotes all possible values in one or more segments, so 1.*.*.* or just 1.* is equivalent to 1.0.0.0/8
  `0-1` denotes the range from 0 to 1
  `_` replaces any digit at the end of a segment, for example 1_ represents 10 to 19 in decimal or 10 to 1f in hex

- **Combinations:** Applying a prefix length to a subnet simply applies the prefix to every element of the subnet. 1.*.0.0/16 is the same subnet as 1.0.0.0/8.

For a more detailed list of formats parsed, some examples are below, or see the javadoc or the godoc for the type `IPAddressString`.

Core Types

The core types are **HostName**, **IPAddressString**, and **MACAddressString** along with the **Address** base type and its subtypes **IPAddress, IPv4Address**, **IPv6Address**, and **MACAddress**, as well as the sequential address types **IPAddressSeqRange**, **IPv4AddressSeqRange** and **IPv6AddressSeqRange**. In Go, the sequential address types are aliases derived from the same generic type.

If you have a textual representation of an IP address, then start with `HostName` or `IPAddressString`. If you have numeric bytes or integers, then start with `IPv4Address`, `IPv6Address` or `MACAddress`. Note that address instances can represent either a single address or a subnet. If you have either an address or host name, or you have something with a port or service name, then use HostName.

Parse String Representation of IP Address, Subnet or Host Name

`IPAddressString` is used to convert.

With the Java library, you can use one of `getAddress` or `toAddress`, the difference being whether parsing errors are handled by exception or not.

```
IPAddress address = new IPAddressString("1.2.3.4").getAddress();
if(address != null) {
```

```
    //use address here
  }
```

or

```
  String str = "1.2.3.4";
  try {
    IPAddress address = new IPAddressString(str).toAddress();
    //use address here
  } catch (AddressStringException e) {
    String msg = e.getMessage();//detailed message indicating issue
  }
```

If you have either a host name or an address, you can use HostName:

```
  checkHost(new HostName("[::1]"));
  checkHost(new HostName("*"));
  checkHost(new HostName("a.b.com"));

  static void checkHost(HostName host) {
    if(host.isAddress()) {
      System.out.println("address: " +
        host.asAddress().toCanonicalString());
    } else if(host.isAddressString()) {
      System.out.println("address string with ambiguous address: " +
        host.asAddressString());
    } else {
      System.out.println("host name with labels: " +
        Arrays.asList(host.getNormalizedLabels()));
    }
  }
```

Output:

```
  address: ::1
  address string with ambiguous address: *
  host name with labels: [a, b, com]
```

Similarly, with the Go library, you can use one of `GetAddress` or `ToAddress`, the difference being whether parsing errors are detected by checking for nil or checking an error.

```
  var address *IPAddress = ipaddr.NewIPAddressString("1.2.3.4").GetAddress()
  if address != nil {
          //use address here
  }
```

or

```
str := "1.2.3.4"
address, err := ipaddr.NewIPAddressString(str).ToAddress()
if err != nil {
        msg := err.Error() //detailed message indicating issue
} else {
        //use address here
}
```

If you have either a host name or an address, you can use HostName:

```
checkHost(ipaddr.NewHostName("[::1]"))
checkHost(ipaddr.NewHostName("*"))
checkHost(ipaddr.NewHostName("a.b.com"))

func checkHost(host *ipaddr.HostName) {
        if host.IsAddress() {
                fmt.Println("address:",
                        host.AsAddress().ToCanonicalString())
        } else if host.IsAddressString() {
                fmt.Println("address string with ambiguous address:",
                        host.AsAddressString())
        } else {
                fmt.Println("host name with labels:",
                        host.GetNormalizedLabels())
        }
}
```

Output:

```
address: ::1
address string with ambiguous address: *
host name with labels: [a b com]
```

Format Examples

Many formats are supported. For instance, the address `1:2:3:0:0:6::` can be represented many ways as shown with this Java code:

```
static void parse(String formats[]) {
  for(String format : formats) {
    System.out.println(new IPAddressString(format).getAddress());
```

```java
    }
  }

  static void parseHost(String formats[]) {
    for(String format : formats) {
      System.out.println(new HostName(format).getAddress());
    }
  }

  String formats[] = {
    "1:2:3:0:0:6::",
    "1:2:3:0:0:6:0:0",
    "1:2:3::6:0:0",
    "0001:0002:0003:0000:0000:0006:0000:0000",
    "1:2:3::6:0.0.0.0",
    "1:2:3:0:0:6::",
    "0b0000000000000001:0b0000000000000010:0b0000000000000011:0:0:0b0000000000000110::",
    "008JQWOV7O(=61h*;$LC",
    "0x00010002000300000000000600000000"
  };
  parse(formats);

  String hostFormats[] = {
    "[1:2:3:0:0:6::]",
    "[1:2:3:0:0:6:0:0]",
    "[1:2:3::6:0:0]",
    "[0001:0002:0003:0000:0000:0006:0000:0000]",
    "[1:2:3::6:0.0.0.0]",
    "[1:2:3:0:0:6::]",
    "[0b0000000000000001:0b0000000000000010:0b0000000000000011:0:0:0b0000000000000110::]",
    "[008JQWOV7O(=61h*;$LC]",
    "[0x00010002000300000000000600000000]",
    "0.0.0.0.0.0.0.0.6.0.0.0.0.0.0.0.0.0.0.0.3.0.0.0.0.2.0.0.0.1.0.0.0.ip6.arpa",
    "1-2-3-0-0-6-0-0.ipv6-literal.net"
  };
  parseHost(hostFormats);
```

or this equivalent Go code:

```go
  func parse(formats []string) {
          for _, format := range formats {
                  fmt.Println(ipaddr.NewIPAddressString(format).GetAddress())
          }
  }

  func parseHost(formats []string) {
          for _, format := range formats {
                  fmt.Println(ipaddr.NewHostName(format).GetAddress())
          }
  }
```

```go
formats := []string{
        "1:2:3:0:0:6::",
        "1:2:3:0:0:6:0:0",
        "1:2:3::6:0:0",
        "0001:0002:0003:0000:0000:0006:0000:0000",
        "1:2:3::6:0.0.0.0",
        "1:2:3:0:0:6::",
        "0b0000000000000001:0b0000000000000010:0b0000000000000011:0:0:0b0000000000000110::",
        "008JQWOV7O(=61h*;$LC",
        "0x00010002000300000000000600000000",
}
parse(formats)

hostFormats := []string{
        "[1:2:3:0:0:6::]",
        "[1:2:3:0:0:6:0:0]",
        "[1:2:3::6:0:0]",
        "[0001:0002:0003:0000:0000:0006:0000:0000]",
        "[1:2:3::6:0.0.0.0]",
        "[1:2:3:0:0:6::]",
        "[0b0000000000000001:0b0000000000000010:0b0000000000000011:0:0:0b0000000000000110::]",
        "[008JQWOV7O(=61h*;$LC]",
        "[0x00010002000300000000000600000000]",
        "0.0.0.0.0.0.0.0.6.0.0.0.0.0.0.0.0.0.0.0.3.0.0.0.2.0.0.0.1.0.0.0.ip6.arpa",
        "1-2-3-0-0-6-0-0.ipv6-literal.net",
}
```

Output from both the Java and Go code:

```
1:2:3::6:0:0
1:2:3::6:0:0
1:2:3::6:0:0
1:2:3::6:0:0
1:2:3::6:0:0
1:2:3::6:0:0
1:2:3::6:0:0
1:2:3::6:0:0
1:2:3::6:0:0
1:2:3::6:0:0
1:2:3::6:0:0
1:2:3::6:0:0
1:2:3::6:0:0
1:2:3::6:0:0
1:2:3::6:0:0
1:2:3::6:0:0
```

Subnet strings are supported as well, using CIDR prefix notation or characters indicating range ('-' for a specific range or '*' for full-range segments).

For instance, the subnet ffff::/104 can be represented many ways.

Here is Java code parsing some of those representations:

```
static void parseSubnet(String formats[]) {
  for(String format : formats) {
    System.out.println(new
    IPAddressString(format).getAddress().assignPrefixForSingleBlock());
  }
}

static void parseHostSubnet(String formats[]) {
  for(String format : formats) {
    System.out.println(new
      HostName(format).getAddress().assignPrefixForSingleBlock());
    }
}


String prefixedFormats[] = {
  "ffff::/104",
  "ffff:0:0:0:0:0:0:0/104",
  "ffff:0000:0000:0000:0000:0000:0000:0000/104",
  "ffff::/104",
  "ffff::0.0.0.0/104",
  "0b1111111111111111::/104",
  "=q{+M|w0(OeO5^EGP660/104"
};

String rangeFormats[] = {
  "ffff:0:0:0:0:0:0-ff:*",
  "ffff::0-ff:*",
  "0xffff0000000000000000000000000000-0xffff0000000000000000000000ffffff"
};

parseSubnet(prefixedFormats);

parseSubnet(rangeFormats);

String hostFormats[] = {
  "[ffff::]/104",
  "[ffff:0:0:0:0:0:0:0]/104",
  "[ffff:0000:0000:0000:0000:0000:0000:0000]/104",
  "[ffff::]/104",
  "[ffff::0.0.0.0]/104",
  "[0b1111111111111111::]/104",
  "[=q{+M|w0(OeO5^EGP660]/104",
  "[ffff:0:0:0:0:0:0-ff:*]",
  "[ffff::0-ff:*]",
  "[0xffff0000000000000000000000000000-0xffff0000000000000000000000ffffff]",
```

```
  "*.*.*.*.*.*.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.f.f.f.f.ip6.arpa",
  "ffff-0-0-0-0-0-0-0.ipv6-literal.net/104"
};

parseHostSubnet(hostFormats);
```

Here is Go code parsing the same string representations:

```
func parseSubnet(formats []string) {
        for _, format := range formats {
                fmt.Println(ipaddr.NewIPAddressString(format).GetAddress().AssignPrefixForSing
        }
}

func parseHostSubnet(formats []string) {
        for _, format := range formats {
                fmt.Println(ipaddr.NewHostName(format).GetAddress().AssignPrefixForSingleBlock
        }
}

prefixedFormats := []string{
        "ffff::/104",
        "ffff:0:0:0:0:0:0:0/104",
        "ffff:0000:0000:0000:0000:0000:0000:0000/104",
        "ffff::/104",
        "ffff::0.0.0.0/104",
        "0b1111111111111111::/104",
        "=q{+M|w0(OeO5^EGP660/104",
}

rangeFormats := []string{
        "ffff:0:0:0:0:0:0-ff:*",
        "ffff::0-ff:*",
        "0xffff000000000000000000000000000-0xffff000000000000000000000ffffff",
}

parseSubnet(prefixedFormats)

parseSubnet(rangeFormats)

hostFormats := []string{
        "[ffff::]/104",
        "[ffff:0:0:0:0:0:0:0]/104",
        "[ffff:0000:0000:0000:0000:0000:0000:0000]/104",
        "[ffff::]/104",
        "[ffff::0.0.0.0]/104",
        "[0b1111111111111111::]/104",
        "[=q{+M|w0(OeO5^EGP660]/104",
        "[ffff:0:0:0:0:0:0-ff:*]",
        "[ffff::0-ff:*]",
        "[0xffff000000000000000000000000000-0xffff000000000000000000000ffffff]",
```

```
        "*.*.*.*.*.*.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.f.f.f.f.ip6.arpa",
        "ffff-0-0-0-0-0-0-0.ipv6-literal.net/104",
}
```

Note that the parsing code is the same for subnets as addresses. The additional call to assign the prefix corresponding to a single block (`assignPrefixForSingleBlock`) simply ensures a consistent prefix length in the final result.

Output from both the Java and Go code:

```
ffff::/104
ffff::/104
ffff::/104
ffff::/104
ffff::/104
ffff::/104
ffff::/104
ffff::/104
ffff::/104
ffff::/104
ffff::/104
ffff::/104
ffff::/104
ffff::/104
ffff::/104
ffff::/104
ffff::/104
ffff::/104
ffff::/104
ffff::/104
ffff::/104
ffff::/104
ffff::/104
```

Delimited Segments

The subnet formats allow you to specify ranges of values. However, if you wish to parse addresses in which the segment values are delimited, there are methods provided to do that.

In Java, you can use the methods `parseDelimitedSegments` and `countDelimitedAddresses` of `IPAddressString` . The former method will provide an iterator to traverse through the individual addresses, while the latter will provide the number of iterated elements.

In Go, the type `DelimitedAddressString` provides this functionality. It has the method `ParseDelimitedSegments` to parse, and `CountDelimitedAddresses` to count the number of delimited

address strings. `ParseDelimitedSegments` will return an iterator which can be used to iterate through each string, parsing each one at a time.

For example, parsing the delimited segments of `"1,2.3.4,5.6"` will iterate through `"1.3.4.6"`, `"1.3.5.6"`, `"2.3.4.6"` and `"2.3.5.6"`. You can construct `IPAddressString` instances from each individual string.

Address or Host Name Validation Options

Validation parameters allow you to restrict the permitted string formats, whether you wish to support just IPv4 or IPv6, or whether you wish to support just single addresses, or whether you wish to allow different address or subnet variants.

For IP addresses you can use `IPAddressStringParameters` with `IPAddressString`, and for host names you can use `HostNameParameters` with `HostName`.

Builders are used to construct. The following example Java code host options along with nested address options within:

```
HostNameParameters HOST_OPTIONS_EXAMPLE = new HostNameParameters.Builder().
        allowEmpty(false).
        setNormalizeToLowercase(true).
        allowBracketedIPv6(true).
        allowBracketedIPv4(true).
  getAddressOptionsBuilder().
    allowPrefix(true).
    allowMask(true).
    setRangeOptions(RangeParameters.WILDCARD_AND_RANGE).
    allow_inet_aton(true).
    allowEmpty(false).
    allowAll(false).
    allowPrefixOnly(false).
    getIPv4AddressParametersBuilder().
      allowPrefixLengthLeadingZeros(true).
      allowPrefixesBeyondAddressSize(false).
      allowWildcardedSeparator(true).
      getParentBuilder().
    getParentBuilder().
  toParams();
```

This is the equivalent code for Go. The parameters code for Go exists in a sub-package, the addrstrparam package.

```
var hostOptionsExample addrstrparam.HostNameParams = new(addrstrparam.HostNameParamsBuilder).
        AllowEmpty(false).
```

```
        NormalizeToLowercase(true).
        AllowBracketedIPv6(true).
        AllowBracketedIPv4(true).
        GetIPAddressParamsBuilder().
                AllowPrefix(true).
                AllowMask(true).
                SetRangeParams(addrstrparam.WildcardAndRange).
                Allow_inet_aton(true).
                AllowEmpty(false).
                AllowAll(false).
                GetIPv4AddressParamsBuilder().
                        AllowPrefixLenLeadingZeros(true).
                        AllowPrefixesBeyondAddressSize(false).
                        AllowWildcardedSeparator(true).
                        GetParentBuilder().
                GetParentBuilder().
```

The default options used by the library are permissive and not restrictive.

Host Name or Address with Port or Service Name

For an address or host with port or service name, use `HostName`. IPv6 addresses with ports should appear as `[ipv6Address]:port` to resolve the ambiguity of the colon separator, consistent with RFC 2732, 3986, 4038 and other RFCs. However, this library will parse IPv6 addresses without the brackets. You can use the "expectPort" setting of `HostNameParameters` to resolve ambiguities when the brackets are absent.

Java example code:

```java
HostName hostName = new HostName("[::1]:80");
System.out.println("host: " + hostName.getHost() + " address: " +
  hostName.getAddress() + " port: " + hostName.getPort());

hostName = new HostName("localhost:80");
System.out.println("host: " + hostName.getHost() + " port: " +
  hostName.getPort());

hostName = new HostName("127.0.0.1:80");
System.out.println("host: " + hostName.getHost() + " address: "
  + hostName.getAddress() + " port: " + hostName.getPort());
```

Go example code:

```go
hostName := ipaddr.NewHostName("[::1]:80")
fmt.Println("host:", hostName.GetHost(), "address:",
        hostName.GetAddress(), "port:", hostName.GetPort())
```

```
hostName = ipaddr.NewHostName("localhost:80")
fmt.Println("host:", hostName.GetHost(), "port:",
        hostName.GetPort())

hostName = ipaddr.NewHostName("127.0.0.1:80")
fmt.Println("host:", hostName.GetHost(), "address:",
        hostName.GetAddress(), "port:", hostName.GetPort())
```

Output from both the Java and Go code:

```
host: ::1 address: ::1 port: 80
host: localhost port: 80
host: 127.0.0.1 address: 127.0.0.1 port: 80
```

IP Version Determination and IPv4/v6 Conversion

With an `IPAddress` or `IPAddressString` object, you can check the version with `isIPv4()` / `IsIPv4()` and `isIPv6()` / `IsIPv6()`. With an `IPAddress`, you can obtain the more specific type corresponding to the IP version, either `IPv4Address` or `IPv6Address`, by calling `toIPv4()` / `ToIPv4()` or `toIPv6()` / `ToIPv6()`, which will return the more specific type if the original address was constructed as that type. Java code:

```
IPv6Address addr6 = new IPAddressString("2001:0db8:85a3:0000:0000:8a2e:0370:7334").getAddress(
```

Go code:

```
var addr6 *ipaddr.IPv6Address
addr6 = ipaddr.NewIPAddressString("2001:0db8:85a3:0000:0000:8a2e:0370:7334").GetAddress().ToIP
```

In Java, you can make use of `isIPv4Convertible()` and `isIPv6Convertible()` to do further conversions if the address is IPv4-mapped.

```
IPAddressString str = new IPAddressString("::ffff:1.2.3.4");
if(str.isIPv6()) {
  IPv6Address ipv6Address = str.getAddress().toIPv6();
  System.out.println(ipv6Address.toMixedString());
  if(ipv6Address.isIPv4Convertible()) {
    IPv4Address ipv4Address = ipv6Address.toIPv4();
    System.out.println(ipv4Address.toNormalizedString());
```

```
    }
  }
```

Output:

```
::ffff:1.2.3.4
1.2.3.4
```

In Java, should you wish to change the default IPv4/IPv6 conversions from IPv4 mapped to something else, you can override the pair of methods `toIPv4()` and `isIPv4Convertible()` in your own `IPv6Address` subclass and/or the pair of methods `toIPv6()` and `isIPv6Convertible()` in your own `IPv4Address` subclass.

The Go library does not allow for implicit and automatic conversion. Much like the Go language in general, the code must be more explicit.

In both Go and Java, you can use your own instance of `IPAddressConverter` for some other suitable conversion, such as IPv4-translated, 6to4, 6over4, IPv4-compatible, or other. The wiki code examples provide example code for Java conversions and Go conversions for all of those IPv4/v6 conversion protocols.

Parsing Addresses with Prefix Length

This library will parse CIDR prefix IP addresses such as 10.1.2.3/24. That string can be interpreted both as an individual address, or as a prefix block with prefix 10.1.2. With other libraries, the ambiguity is resolved by the method, function, or type used for parsing. However, this library uses the same types and methods for both subnets and individual addresses, which can result in much cleaner and more flexible code, but also requires that such ambiguities are resolved differently.

Network addresses (addresses with a host that is zero) like 10.1.2.0/24 and a:b:c:d::/64 are parsed as the full block of addresses with the indicated prefix - the network prefix remains constant while the host spans all values. This subnet is called the prefix block. If the host is not zero, such as with 10.1.2.3/24, then the string is parsed as a single individual address, but with an associated prefix length. This convention applies to both IPv4 and IPv6.

The same rule applies to strings in which the address is a subnet in which the subnet lower and upper boundaries have zero hosts, like 10.1.2-3.0/24 or 10.1.2.2-6/31. Both of those examples will become CIDR prefix blocks when parsed or constructed, the first with 512 addresses, and the second with 10 addresses.

When parsing an IPAddress string, you can ignore the presence of a prefix length or mask in the string with `getHostAddress` / `GetHostAddress` or `toHostAddress` / `ToHostAddress`. You will get the host address 10.1.2.3 when parsing 10.1.2.3/24. The prefix length remains available by calling `getNetworkPrefixLength` in Java, or `GetNetworkPrefixLen` in Go, on the `IPAddressString` instance.

The example code below shows some of these methods in use. Take note of the count for each parsed string, indicating whether it represents a subnet or address.

```java
static void printPrefixedAddresses(String addressStr) {
  IPAddressString ipAddressString = new IPAddressString(addressStr);
  IPAddress address = ipAddressString.getAddress();
  System.out.println("count: " + address.getCount());
  IPAddress hostAddress = ipAddressString.getHostAddress();
  IPAddress prefixBlock = address.toPrefixBlock();
  Integer prefixLength = ipAddressString.getNetworkPrefixLength();
  System.out.println(address);
  System.out.println(address.toCanonicalWildcardString());
  System.out.println(hostAddress);
  System.out.println(prefixLength);
  System.out.println(prefixBlock);
  System.out.println();
}

printPrefixedAddresses("10.1.2.3/24"); // individual address
printPrefixedAddresses("10.1.2.0/24"); // network
```

The equivalent code for Go is:

```go
func printPrefixedAddresses(addressStr string) {
        ipAddressString := ipaddr.NewIPAddressString(addressStr)
        address := ipAddressString.GetAddress()
        fmt.Println("count:", address.GetCount())
        hostAddress := ipAddressString.GetHostAddress()
        prefixBlock := address.ToPrefixBlock()
        prefixLength := ipAddressString.GetNetworkPrefixLen()
        fmt.Println(address)
        fmt.Println(address.ToCanonicalWildcardString())
        fmt.Println(hostAddress)
        fmt.Println(prefixLength)
        fmt.Println(prefixBlock)
        fmt.Println()
}

printPrefixedAddresses("10.1.2.3/24") // individual address
printPrefixedAddresses("10.1.2.0/24") // network
```

Output from both the Java and Go code:

```
count: 1
10.1.2.3/24
10.1.2.3
10.1.2.3
```

```
24
10.1.2.0/24

count: 256
10.1.2.0/24
10.1.2.*
10.1.2.0
24
10.1.2.0/24
```

Parse Non-Segmented Addresses – Hex, Octal, IPv6 Base 85, Binary

Typically, the segments or other punctuation identify a string as a host name, as an IPv4 address, or as an IPv6 address. The parser also parses single segment values, or a range of single segment values.

With non-segmented addresses, ambiguity between IPv4 and IPv6 is resolved by the number of digits in the string. The number of digits is 32 for IPv6 hexadecimal, 20 for IPv6 base 85 (see RFC 1924), and 11 or less for IPv4, which can be octal, hexadecimal, or decimal. For IPv4, digits are presumed decimal unless preceded by 0x for hexadecimal or 0 for octal, as is consistent with the inet_aton routine. For IPv6, 32 digits are considered hexadecimal and a preceding 0x is optional.

Here is some Java code parsing single-segment addresses:

```java
IPAddressString ipAddressString = new IPAddressString("4)+k&C#VzJ4br>0wv%Yp"); // base 85
IPAddress address = ipAddressString.getAddress();
System.out.println(address);

ipAddressString = new IPAddressString("1080000000000000000080800200c417a"); // hex IPv6
address = ipAddressString.getAddress();
System.out.println(address);

ipAddressString = new IPAddressString("0b00010000100000000000000000000000000000000000000000000000
address = ipAddressString.getAddress();
System.out.println(address);

ipAddressString = new IPAddressString("0x01020304"); // hex IPv4
address = ipAddressString.getAddress();
System.out.println(address);

ipAddressString = new IPAddressString("000100401404"); // octal IPv4
address = ipAddressString.getAddress();
System.out.println(address);

ipAddressString = new IPAddressString("0b00000001000000100000001100000100"); // binary IPv4
```

```
address = ipAddressString.getAddress();
System.out.println(address);
```

Here is equivalent Go code parsing the same single-segment addresses:

```
ipAddressString := ipaddr.NewIPAddressString("4)+k&C#VzJ4br>0wv%Yp") // base 85
address := ipAddressString.GetAddress()
fmt.Println(address)

ipAddressString = ipaddr.NewIPAddressString("1080000000000000000080800200c417a") // hex IPv6
address = ipAddressString.GetAddress()
fmt.Println(address)

ipAddressString = ipaddr.NewIPAddressString("0b0001000010000000000000000000000000000000000000
address = ipAddressString.GetAddress()
fmt.Println(address)

ipAddressString = ipaddr.NewIPAddressString("0x01020304") // hex IPv4
address = ipAddressString.GetAddress()
fmt.Println(address)

ipAddressString = ipaddr.NewIPAddressString("000100401404") // octal IPv4
address = ipAddressString.GetAddress()
fmt.Println(address)

ipAddressString = ipaddr.NewIPAddressString("0b00000001000000100000001100000100") // binary IP
address = ipAddressString.GetAddress()
fmt.Println(address)
```

Output from both the Java and Go code:

```
1080::8:800:200c:417a
1080::8:800:200c:417a
1080::8:800:200c:417a
1.2.3.4
1.2.3.4
1.2.3.4
```

When parsing a range of single-segment values, it might not be possible to represent the range as a series of segments of range values, which is what is needed to be represented by an `IPv6Address` of 8 segment ranges, or an `IPv4Address` of 4 segment ranges.

However, the string can still be parsed. In Java, the parsed result can be obtained using `toDivisionGrouping`, `getDivisionGrouping`, providing an exact representation of the string divisions. In

both Java and Go, you can call the method `getSequentialRange` or `coverWithSequentialRange` providing an `IPAddressSeqRange` instance with the range of addresses from the lower to the upper value of the range expressed by the string. From the range, a series of `IPAddress` instances can be obtained using `spanWithPrefixBlocks` or `spanWithSequentialBlocks`.

Parse Special Host Names – Reverse DNS Host Name, IPv6 Literal UNC Host Name

A couple of standardized host formats are recognized, namely the reverse DNS host format, and the UNC IPv6 literal host format.

Here is a Java code example parsing such strings:

```
HostName hostName = new HostName("a.7.1.4.c.0.0.2.0.0.8.0.8.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.8.
System.out.println(hostName.asAddress());

hostName = new HostName("4.3.2.1.in-addr.arpa");
System.out.println(hostName.asAddress());

hostName = new HostName("1080-0-0-0-8-800-200c-417a.ipv6-literal.net");
System.out.println(hostName.asAddress());
```

Here is the equivalent Go code:

```
hostName := ipaddr.NewHostName("a.7.1.4.c.0.0.2.0.0.8.0.8.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.8.0.
fmt.Println(hostName.AsAddress())

hostName = ipaddr.NewHostName("4.3.2.1.in-addr.arpa")
fmt.Println(hostName.AsAddress())

hostName = ipaddr.NewHostName("1080-0-0-0-8-800-200c-417a.ipv6-literal.net")
fmt.Println(hostName.AsAddress())
```

Output from both the Java and Go code:

```
1080::8:800:200c:417a
1.2.3.4
1080::8:800:200c:417a
```

A couple of methods in `HostName` are available to indicate such strings:

```java
public boolean isUNCIPv6Literal()
public boolean isReverseDNS()
```

```go
func (host *HostName) IsUncIPv6Literal() bool
func (host *HostName) IsReverseDNS() bool
```

Parse IPv6 Zone or Scope ID

The IPv6 zone or scope ID is recognized, denoted by the '%' character. It can be retrieved by the method `getZone` in `IPv6Address`. Here is sample Java code:

```java
IPAddress addr = new IPAddressString("::%eth0").getAddress();
if(addr.isIPv6()) {
  System.out.println(addr.toIPv6().getZone());
}
```

Here is sample Go code to do the same:

```go
addr := ipaddr.NewIPAddressString("::%eth0").GetAddress()
if addr.IsIPv6() {
        fmt.Println(addr.ToIPv6().GetZone())
}
```

Output from both the Java and Go code:

```
eth0
```

Addresses from Numeric Values

In addition to the range of string formats that can be parsed to produce `IPAddress` instances, you can also obtain `IPAddress` instances from a numerous numeric formats. You an obtain instances of `IPAddress` from byte arrays in Java and from byte slices in Go. You can obtain instances of `IPAddress` from `java.net.InetAddress` or `java.net.InterfaceAddress` in Java and any one of `net.IP`, `net.IPAddr`, `net.IPMask`, `net.IPNet`, `netip.Addr`, or `netip.Prefix` in Go. You can obtain instances of `IPAddress` from arrays of address segments in Java, or from slices of address segments in Go. You can obtain instances of `IPAddress` from individual integer segment values using the `SegmentValueProvider` interface.

All of these options are generic to either IPv4 or IPv6 unless you specifically choose the IPv4 or IPv6-specific constructors. See the godoc or javadoc for the full list.

For IPv4 you have the additional option of constructing an address from a 32-bit integer. See the godoc or javadoc.

For IPv6, you have the additional options of constructing from MAC address instances, from a `java.math.BigInteger` in Java, or from a `math/big.Int` or a pair of 64-bit unsigned integers in Go. See the godoc or javadoc.

Once you have an IPAddress instance, there are methods to convert to bytes, to sections, to subnets or network prefixes, to masks, to all the same standard-library types from which you can construct an IPAddress instance, to different string representations, and so on.

When constructing IP addresses or sections, you can supply a prefix length, and when you do, the same rules regarding zero-hosts applies as when parsing from strings. Network addresses (addresses with host that is zero) like 10.1.2.0/24 and a:b:c:d::/64 are constructed as the prefix block of addresses with the indicated prefix. If the host is not zero, like 10.1.2.3/24, then it is constructed as an individual address, an address with an associated prefix length. This applies to both IPv4 and IPv6.

The same rule applies to subnets where the lower and upper values have zero hosts, like 10.1.2-3.0/24 or 10.1.2.2-6/31. Both of those examples will become CIDR prefix blocks when constructed.

Should you wish to get the individual address or section with a zero host, you can construct without the prefix length and then apply the prefix length afterwards, or you can use `getLower` or `toZeroHost` after construction.

Golang Address Keys

The Go language has the concept of comparable types, those types that can be compared with comparison operators. The core types of this library are not comparable in that manner, although they are all comparable with each other using their `Compare` methods, or using one of the library's comparator instances.

Each of the address and range core types provides an associated key type, a value type, that is comparable with comparison operators and usable as keys for the Go built-in map type. Use the `ToKey` methods to obtain the corresponding key, and use each key's `ToAddress` method to get back the corresponding address.

You can see an example using address keys for sets using maps in the example wiki.

Address, subnet and range keys are also useful for data transfer across networks. They are simple and compact value types (primarily integers) that are ideal for including in protobuf, json, or other network

formats and protocols. Once transmitted they can be easily converted back to the library's default type allowing for complex address and subnet manipulations and transformations.

Golang Zero Values

The following Go code reveals the zero values for the address and sequential range core types, as well as some other related types. As you might expect, the zero values for addresses and segments are zero-valued addresses and segments:

```
strip := func(s string) string {
        return strings.ReplaceAll(strings.ReplaceAll(s, "ipaddr.", ""),
                "github.com/seancfoley/ipaddress-go/", "")
}

typeName := func(i any) string {
        return strip(reflect.ValueOf(i).Elem().Type().Name())
}

interfaceTypeName := func(i any) string {
        return strip(reflect.TypeOf(i).String())
}

truncateIndent := func(s, indent string) string {
        if boundary := len(indent) - (len(s) >> 3); boundary >= 0 {
                return indent[:boundary] + "\t" // every 8 chars eliminates a tab
        }
        return ""
}

baseIndent := "\t\t\t"
title := "Address item zero values"
fmt.Printf("%s%sint\tbits\tcount\tstring\n", title, truncateIndent(title, baseIndent))
vars := []ipaddr.AddressItem{
        &ipaddr.Address{}, &ipaddr.IPAddress{},
        &ipaddr.IPv4Address{}, &ipaddr.IPv6Address{}, &ipaddr.MACAddress{},

        &ipaddr.AddressSection{}, &ipaddr.IPAddressSection{},
        &ipaddr.IPv4AddressSection{}, &ipaddr.IPv6AddressSection{}, &ipaddr.MACAddressSection{
        &ipaddr.EmbeddedIPv6AddressSection{},
        &ipaddr.AddressDivisionGrouping{}, &ipaddr.IPAddressLargeDivisionGrouping{},
        &ipaddr.IPv6v4MixedAddressGrouping{},

        &ipaddr.AddressSegment{}, &ipaddr.IPAddressSegment{},
        &ipaddr.IPv4AddressSegment{}, &ipaddr.IPv6AddressSegment{}, &ipaddr.MACAddressSegment{
        &ipaddr.AddressDivision{}, &ipaddr.IPAddressLargeDivision{},

        &ipaddr.IPAddressSeqRange{}, &ipaddr.IPv4AddressSeqRange{}, &ipaddr.IPv6AddressSeqRang
```

```
}
for _, v := range vars {
        name := typeName(v) + "{}"
        indent := truncateIndent(name, baseIndent)
        fmt.Printf("%s%s%v\t%v\t%v\t\"%v\"\n", name, indent, v.GetValue(), v.GetBitCount(), v.
```

Zero values for versioned address types, like those for IPv4 and IPv6, are the respective zero-valued addresses.

For other addresses, sections and groupings, those with no specific address type, version, and length, the zero values have no segments nor divisions. They have a total of zero bits.

Regardless of bit-size, all zero-values have a corresponding integer value of zero.

The zero values for sequential ranges correspond to ranges with both boundaries as the corresponding zero valued address.

Output:

```
Address item zero values        int     bits    count   string
Address{}                       0       0       1       ""
IPAddress{}                     0       0       1       ""
IPv4Address{}                   0       32      1       "0.0.0.0"
IPv6Address{}                   0       128     1       "::"
MACAddress{}                    0       48      1       "00:00:00:00:00:00"
AddressSection{}                0       0       1       ""
IPAddressSection{}              0       0       1       ""
IPv4AddressSection{}            0       0       1       ""
IPv6AddressSection{}            0       0       1       ""
MACAddressSection{}             0       0       1       ""
EmbeddedIPv6AddressSection{}    0       0       1       ""
AddressDivisionGrouping{}       0       0       1       ""
IPAddressLargeDivisionGrouping{}0       0       1       ""
IPv6v4MixedAddressGrouping{}    0       0       1       ""
AddressSegment{}                0       0       1       "0x0"
IPAddressSegment{}              0       0       1       "0x0"
IPv4AddressSegment{}            0       8       1       "0"
IPv6AddressSegment{}            0       16      1       "0x0"
MACAddressSegment{}             0       8       1       "0x0"
AddressDivision{}               0       0       1       "0x0"
IPAddressLargeDivision{}        0       0       1       "0x0"
SequentialRange[*IPAddress]{}   0       0       1       " -> "
SequentialRange[*IPv4Address]{} 0       32      1       "0.0.0.0 -> 0.0.0.0"
SequentialRange[*IPv6Address]{} 0       128     1       ":: -> ::"
```

Address items allow for producing strings and counts from nil pointers. Most other methods, methods that require analysis of the internals of those struct types, will panic on nil pointers. But the String and GetCount methods will return a string indicating nil and a count of zero.

```go
title = "Address item nil pointers"
fmt.Printf("\n%s%scount\tstring\n", title, truncateIndent(title, baseIndent+"\t\t"))
nilPtrItems := []ipaddr.AddressItem{
        (*ipaddr.Address)(nil), (*ipaddr.IPAddress)(nil),
        (*ipaddr.IPv4Address)(nil), (*ipaddr.IPv6Address)(nil), (*ipaddr.MACAddress)(nil),

        (*ipaddr.AddressSection)(nil), (*ipaddr.IPAddressSection)(nil),
        (*ipaddr.IPv4AddressSection)(nil), (*ipaddr.IPv6AddressSection)(nil), (*ipaddr.MACAddr

        (*ipaddr.AddressSegment)(nil), (*ipaddr.IPAddressSegment)(nil),
        (*ipaddr.IPv4AddressSegment)(nil), (*ipaddr.IPv6AddressSegment)(nil), (*ipaddr.MACAddr

        (*ipaddr.IPAddressSeqRange)(nil), (*ipaddr.IPv4AddressSeqRange)(nil), (*ipaddr.IPv6Add
}
for _, v := range nilPtrItems {
        name := "(" + interfaceTypeName(v) + ")(nil)"
        indent := truncateIndent(name, baseIndent+"\t\t")
        fmt.Printf("%s%s%v\t\"%v\"\n", name, indent, v.GetCount(), v)
}
```

Output:

```
Address item nil pointers                       count   string
(*Address)(nil)                                 0       "<nil>"
(*IPAddress)(nil)                               0       "<nil>"
(*IPv4Address)(nil)                             0       "<nil>"
(*IPv6Address)(nil)                             0       "<nil>"
(*MACAddress)(nil)                              0       "<nil>"
(*AddressSection)(nil)                          0       "<nil>"
(*IPAddressSection)(nil)                        0       "<nil>"
(*IPv4AddressSection)(nil)                      0       "<nil>"
(*IPv6AddressSection)(nil)                      0       "<nil>"
(*MACAddressSection)(nil)                       0       "<nil>"
(*AddressSegment)(nil)                          0       "<nil>"
(*IPAddressSegment)(nil)                        0       "<nil>"
(*IPv4AddressSegment)(nil)                      0       "<nil>"
(*IPv6AddressSegment)(nil)                      0       "<nil>"
(*MACAddressSegment)(nil)                       0       "<nil>"
(*SequentialRange[*IPAddress])(nil)             0       "<nil>"
(*SequentialRange[*IPv4Address])(nil)           0       "<nil>"
(*SequentialRange[*IPv6Address])(nil)           0       "<nil>"
```

The address key value types have zero values matching the zero values of their corresponding address and range types.

```go
title = "Address key zero values"
fmt.Printf("\n%s%sstring\n", title, truncateIndent(title, baseIndent+"\t\t\t"))
keys := []fmt.Stringer{
        &ipaddr.AddressKey{}, &ipaddr.IPAddressKey{},
        &ipaddr.IPv4AddressKey{}, &ipaddr.IPv6AddressKey{}, &ipaddr.MACAddressKey{},
        &ipaddr.IPAddressSeqRangeKey{}, &ipaddr.IPv4AddressSeqRangeKey{}, &ipaddr.IPv6AddressS
}
for _, k := range keys {
        name := typeName(k) + "{}"
        indent := truncateIndent(name, baseIndent+"\t\t\t")
        fmt.Printf("%s%s\"%v\"\n", name, indent, k)
}
```

Output:

```
Address key zero values                                    string
Key[*Address]{}                                            ""
Key[*IPAddress]{}                                          ""
IPv4AddressKey{}                                           "0.0.0.0"
IPv6AddressKey{}                                           "::"
MACAddressKey{}                                            "00:00:00:00:00:00"
SequentialRangeKey[*IPAddress]{}                           " -> "
SequentialRangeKey[*IPv4Address]{}                         "0.0.0.0 -> 0.0.0.0"
SequentialRangeKey[*IPv6Address]{}                         ":: -> ::"
```

The zero values of host identifier strings are empty strings.

```go
title = "Host id zero values"
fmt.Printf("\n%s%sstring\n", title, truncateIndent(title, baseIndent+"\t\t\t"))
hostids := []ipaddr.HostIdentifierString{
  &ipaddr.HostName{}, &ipaddr.IPAddressString{}, &ipaddr.MACAddressString{},
}
for _, k := range hostids {
  name := typeName(k) + "{}"
  indent := truncateIndent(name, baseIndent+"\t\t\t")
  fmt.Printf("%s%s\"%v\"\n", name, indent, k)
}
```

Output:

```
Host id zero values                                        string
HostName{}                                                 ""
IPAddressString{}                                          ""
MACAddressString{}                                         ""
```

Like addresses, host identifier strings allow for producing strings from nil pointers. Most other methods of these types will panic on nil pointers, but the String methods will return a string indicating nil.

```
title = "Host id nil pointers"
fmt.Printf("\n%s%sstring\n", title, truncateIndent(title, baseIndent+"\t\t\t"))
nilPtrIds := []ipaddr.HostIdentifierString{
  (*ipaddr.HostName)(nil), (*ipaddr.IPAddressString)(nil), (*ipaddr.MACAddressString)(nil),
}
for _, v := range nilPtrIds {
  name := "(" + interfaceTypeName(v) + ")(nil)"
  indent := truncateIndent(name, baseIndent+"\t\t\t")
  fmt.Printf("%s%s\"%v\"\n", name, indent, v)
}
```

Output:

```
Host id nil pointers                                        string
(*HostName)(nil)                                            "<nil>"
(*IPAddressString)(nil)                                     "<nil>"
(*MACAddressString)(nil)                                    "<nil>"
```

Networks

Each of the IP address versions have an associated singleton network object. The network objects are used for caching, for configuration, or for obtaining masks and loopbacks.

Each of the IP address versions also have an associated “creator” type that can be used to create addresses, sections, and segments, and instances of those types may perform caching of address components for efficient memory usage and performance.

In the Java library, the methods `defaultIpv6Network` and `defaultIpv4Network` in Address provide access to the respective network objects. There is also a counterpart for MAC, available from the `defaultMACNetwork` method. Each network has an associated creator object available from the `getAddressCreator` method.

In the Go library, the network objects are the package-level variables `ipaddr.IPv4Network` and `ipaddr.IPv6Network`. Use the type IPAddressCreator for creator instances.

Prefix Length Handling

Prefix lengths in strings are parsed, as indicated in the section above on parsing, or can be supplied when directly constructing addresses or sections. Addresses and sections store their prefix lengths and the prefix length is incorporated in numerous address operations as well as when producing strings. For instance, an address will provide the network section, based upon the prefix length, with calls to the IPAddress method `getNetworkSection` in Java or `GetNetworkSection` in Go, and will supply the prefix length when calling `getNetworkPrefixLength` in Java or `GetNetworkPrefixLen` in Go.

Given an address with no prefix length, you can convert to an address with prefix length using the methods `assignPrefixForSingleBlock` / `AssignPrefixForSingleBlock` or `assignMinPrefixForBlock` / `AssignMinPrefixForBlock` , or any of the methods that allow you to set a prefix length directly such as `setPrefixLength` / `SetPrefixLen` or `adjustPrefixLength` / `AdjustPrefixLen` in Java / Go.

Anytime you have an individual address or a subnet with prefix length, you can get the address representing the entire block for that prefix using the method `toPrefixBlock` / `ToPrefixBlock` in in Java / Go. This type of subnet can be called a CIDR prefix block, a subnet which spans all the hosts for a specific CIDR prefix. In the reverse direction, given a CIDR prefix block, you can get the IPv4 network address or the IPv6 anycast address by calling `getLower` / `GetLower` or by calling `toZeroHost` / `ToZeroHost` .

Prefix Length and Equality

In this library, the subnet with prefix length 10.1.2.0/24, which can also be written as 10.1.2.*/24, is equivalent the non-prefixed address 10.1.2.*, since they both contain the same set of numeric addresses. In other words, when it comes to equality or comparison, the prefix length has no effect. Equality and comparison is entirely based on numeric values.

Address Sections

Addresses can be broken up into sections, and reconstituted from sections. A section is a series of segments. You can get the section for the full address by calling `getSection` , or you can get subsections by calling one of the variants, either `getSection(int)` or `getSection(int, int)` in Java. In Go you would use `GetSubSection` or `GetTrailingSection` . These methods return a subsection spanning the given indices. You can also get the segments in an address by calling `getSegment` or one of the variants of `getSegments` in Java, or one of `GetSegments` , `CopySegments` or `CopySubSegments` in Go, either on the address or on a section of the address.

You can also reconstitute an address from a section or array of segments using the appropriate address constructor, if your section or array of segments has the correct number of segments for the address type.

Host and Network Sections of IP Address

Use `getHostSection()` and `getNetworkSection()` to get the host and network sections of an IP address as indicated by prefix length, as shown by this Java code:

```
IPAddress address = new IPAddressString("1.2.3.4").getAddress();
IPAddressSection network = address.getNetworkSection(16, true);
IPAddressSection host = address.getHostSection(16);
System.out.println(network.toCanonicalString());
System.out.println(host.toCanonicalString());
```

This is the equivalent Go code:

```
address := ipaddr.NewIPAddressString("1.2.3.4").GetAddress()
network := address.GetNetworkSectionLen(16)
host := address.GetHostSectionLen(16).WithoutPrefixLen()
fmt.Println(network.ToCanonicalString())
fmt.Println(host.ToCanonicalString())
```

Output from both the Java and Go code:

```
1.2/16
3.4
```

Once you have a section of an address, most of the same methods are available as those available with addresses themselves.

IP Address Ranges

IPAddress Internal Format

An `IPAddress` or `IPAddressString` instance can represent any individual address or any range of addresses in which each segment specifies a range of sequential values. Such a range can be called a segment block, a block of values within a given segment's range of possible values. A subnet that contains segment blocks for any of its segments can be called a segment block subnet, or a block subnet. An individual address is one in which each segment is a segment block of size one, just a single value.

Any CIDR prefix block can be specified as segment blocks. However, a prefix block is a more specific type of subnet, a subnet that contains the full range of values for its prefix. So any segment outside the prefix is a

segment block containing all possible values for the segment.

An `IPAddress` instance has the canonical number of segments for its address version or type, which is 4 segments for IPv4, 8 for IPv6, and either 6 or 8 for MAC.

An `IPAddressString` can represent any such address string, as well as those that do not use the canonical number of segments. However, for ranges that do not have the canonical number of segments, converting to an `IPAddress` instance is not always possible (for example the IPv4 subnet 1.2-3.0-1000 cannot be expressed with 4 segments).

Sequential Blocks

Not all `IPAddress` subnets are sequential. For instance, 1-2.3.4-5.6 is not sequential, since the address 1.3.4.6 is followed in the block by 1.3.5.6 while the next sequential address 1.3.4.7 is not part of the block. A sequential block is an `IPAddress` subnet in which the range is sequential. For a block to be sequential, the first segment with a range of values must be followed only by segments that cover all values. For instance, 1.2.3-4.* is a sequential block, as well as 1:a-f:*:*:*:*:*:*, also writeable as 1:a-f:*. Any prefix block is sequential. A prefix block can also be expressed without the prefix length. For instance, the prefix block 1:2:3:4::/64 can be written without the prefix length as 1:2:3:4:*.

You can convert a non-sequential block to a collection of sequential blocks using the `sequentialBlockIterator` / `SequentialBlockIterator` method of `IPAddress`. If you wish to get the count of sequential blocks, use the method `getSequentialBlockCount` / `GetSequentialBlockCount`.

This Java and Go code demonstrates the use of the sequential block iterator.

```
convertNonSequentialBlock("a:b:c:d:1:2-4:3-5:4-6");

static void convertNonSequentialBlock(String string) {
  IPAddressString addrString = new IPAddressString(string);
  IPAddress addr = addrString.getAddress();
  System.out.println("Initial range block is " + addr);
  BigInteger sequentialCount = addr.getSequentialBlockCount();
  System.out.println("Sequential range block count is " + sequentialCount);

  Iterator<? extends IPAddress> iterator = addr.sequentialBlockIterator();
  while(iterator.hasNext()) {
    System.out.println(iterator.next());
  }
}
```

```
convertNonSequentialBlock("a:b:c:d:1:2-4:3-5:4-6")

func convertNonSequentialBlock(str string) {
        addrString := ipaddr.NewIPAddressString(str)
        addr := addrString.GetAddress()
```

```go
        fmt.Println("Initial range block is", addr)
        sequentialCount := addr.GetSequentialBlockCount()
        fmt.Println("Sequential range block count is", sequentialCount)

        iterator := addr.SequentialBlockIterator()
        for iterator.HasNext() {
                fmt.Println(iterator.Next())
        }
}
```

Output from both the Java and Go code:

```
Initial range block is a:b:c:d:1:2-4:3-5:4-6
Sequential range block count is 9
a:b:c:d:1:2:3:4-6
a:b:c:d:1:2:4:4-6
a:b:c:d:1:2:5:4-6
a:b:c:d:1:3:3:4-6
a:b:c:d:1:3:4:4-6
a:b:c:d:1:3:5:4-6
a:b:c:d:1:4:3:4-6
a:b:c:d:1:4:4:4-6
a:b:c:d:1:4:5:4-6
```

IP Address Sequential Ranges

Not all sequential address ranges can be described by an instance of `IPAddress` or `IPAddressString`. One such example is the range of two IPv4 addresses from 1.2.3.255 to 1.2.4.0. One option is to represent the address range with just a single large segment covering the section of the address that has a range of values, such as in an instance of `IPAddressLargeDivisionGrouping`, but this is rather contrived and not recommended.

The recommended and common option is to use an `IPAddressSeqRange` instance, which provides a general representation of address ranges and their associated operations. You can represent any sequential range of addresses with an `IPAddressSeqRange` instance. In Go, `IPAddressSeqRange` is an alias for `SequentialRange[*IPAddress]`.

You can also specify more specific ranges using `IPv4AddressSeqRange` and `IPv6AddressSeqRange` if the polymorphism of `IPAddressSeqRange` is not required. An `IPAddressSeqRange` represents either an IPv4 or IPv6 range, but it cannot represent a mix of IPv4 and IPv6 addresses.

Both `IPAddress` and `IPAddressSeqRange` implement the `IPAddressRange` interface. `IPAddressSeqRange` instances cover all ranges of addresses, while `IPAddress` instances cover all ranges of segments within addresses.
Individual addresses and CIDR prefix blocks can be represented by either type, although `IPAddress` instances are the standard for expressing individual addresses and CIDR prefix blocks.

Any `IPAddressSeqRange` instance can be converted to the minimal list of `IPAddress` sequential blocks or prefix blocks that cover the exact same range of addresses, providing a couple of different ways to go from `IPAddressSeqRange` to `IPAddress` instances. To go in the reverse direction, from `IPAddress` to `IPAddressSeqRange`, you can start with a sequential block iterator to convert any `IPAddress` instance to a series of `IPAddress` sequential blocks. Then you can convert each `IPAddress` sequential block to an `IPAddressSeqRange` with the method `coverWithSequentialRange`. Finally, to eliminate overlap and obtain a minimal list of `IPAddressSeqRange` instances, you can use the `join` method of `IPAddressSeqRange`.

Here we show code examples of a round-trip from a sequential range to a list of sequential or prefix blocks, and then back again, merging the list of blocks back to the original sequential range.

First we show the process in Java code:

```
String address1 = "2:3:ffff:5::", address2 = "2:4:1:5::";
IPAddressString string1 = new IPAddressString(address1), string2 = new IPAddressString(address
IPAddress addr1 = string1.getAddress(), addr2 = string2.getAddress();
IPAddressSeqRange range = addr1.spanWithRange(addr2);
System.out.println("Original sequential range of " +
        range.getCount() + " addresses: " + range);

spanAndMergeSequentialBlocks(range);
spanAndMergePrefixBlocks(range);

static void spanAndMergePrefixBlocks(IPAddressSeqRange range) {
        IPAddress result[] = range.spanWithPrefixBlocks();
        System.out.println("Prefix blocks: " + Arrays.asList(result));
        List<IPAddressSeqRange> rangeList = new ArrayList<>();
        for(IPAddress a : result) {
                rangeList.add(a.coverWithSequentialRange());
        }
        mergeBack(rangeList);
}

static void spanAndMergeSequentialBlocks(IPAddressSeqRange range) {
        IPAddress result[] = range.spanWithSequentialBlocks();
        System.out.println("Sequential blocks: " + Arrays.asList(result));
        List<IPAddressSeqRange> rangeList = new ArrayList<>();
        for(IPAddress a : result) {
                rangeList.add(a.coverWithSequentialRange());
        }
        mergeBack(rangeList);
}

static void mergeBack(List<IPAddressSeqRange> rangeList) {
        IPAddressSeqRange joined[] = IPAddressSeqRange.join(
                rangeList.toArray(new IPAddressSeqRange[rangeList.size()]));
```

```java
        System.out.println("Merged back again: " + Arrays.asList(joined));
}
```

Now we show the process in Go code:

```go
address1, address2 := "2:3:ffff:5::", "2:4:1:5::"
string1, string2 := ipaddr.NewIPAddressString(address1), ipaddr.NewIPAddressString(address2)
addr1, addr2 := string1.GetAddress(), string2.GetAddress()
rng := addr1.SpanWithRange(addr2)
fmt.Println("Original sequential range of", rng.GetCount(), "addresses:", rng)

spanAndMergeSequentialBlocks(rng)
spanAndMergePrefixBlocks(rng)

func spanAndMergePrefixBlocks(rng *ipaddr.IPAddressSeqRange) {
        result := rng.SpanWithPrefixBlocks()
        fmt.Println("Prefix blocks:", commaDelimit(result))
        var rangeList []*ipaddr.IPAddressSeqRange
        for _, a := range result {
                rangeList = append(rangeList, a.ToSequentialRange())
        }
        mergeBack(rangeList)
}

func spanAndMergeSequentialBlocks(rng *ipaddr.IPAddressSeqRange) {
        result := rng.SpanWithSequentialBlocks()
        fmt.Println("Sequential blocks:", commaDelimit(result))
        var rangeList []*ipaddr.IPAddressSeqRange
        for _, a := range result {
                rangeList = append(rangeList, a.ToSequentialRange())
        }
        mergeBack(rangeList)
}

func mergeBack(rangeList []*ipaddr.IPAddressSeqRange) {
        var rng *ipaddr.IPAddressSeqRange
        joined := rng.Join(rangeList...) // can handle nil args, including the receiver
        fmt.Println("Merged back again:", joined)
}

func commaDelimit(slice any) string {
        return strings.ReplaceAll(fmt.Sprint(slice), " ", ", ")
}
```

Output from both the Java and Go code:

```
Original range of size 2417851639229258349412353: 2:3:ffff:5:: -> 2:4:1:5::
Sequential blocks: [2:3:ffff:5-ffff:*:*:*:*, 2:4:0:*:*:*:*:*, 2:4:1:0-4:*:*:*:*, 2:4:1:5::]
Merged back again: [2:3:ffff:5:: -> 2:4:1:5::]
Prefix blocks: [2:3:ffff:5::/64, 2:3:ffff:6::/63, 2:3:ffff:8::/61, 2:3:ffff:10::/60, 2:3:ffff:
Merged back again: [2:3:ffff:5:: -> 2:4:1:5::]
```

Another thing visible from the example above, is that you can generally describe a range with fewer sequential blocks than prefix blocks.

Address Tries

The trie data structure is particularly useful when working with addresses. For that reason this library includes compact binary address tries (aka compact binary prefix tree or binary radix trie, amongst other names). Tries provide efficient re**trie**val operations, hence the name *trie*, but what makes them additionally useful for addresses is the fact that prefix tries are organized by the bits in the prefix of each key, the keys being addresses in this case, which mirrors the way that CIDR subnets and addresses are organized by prefix. So you can use tries for efficient subnet containment checks on many addresses or subnets at once in constant time (such as with a routing table). A trie is useful for efficient lookups, for efficiently dividing and subdividing subnets, for sorting addresses, and for traversing through subnets in different ways.

By associating each trie node with a value, tries can also be used for value lookups in which the keys are addresses. The library provides the “Associative” trie types for this purpose. Every trie type has a corresponding associative type. For instance, there is an `IPv4AddressTrie` and an `IPv4AddressAssociativeTrie` for IPv4 addresses.

Tries can also be used as the backing data structures for maps and sets, as well as the backing data structure for a containment trie collection, which is described later in this document in the section on IP address collections.

With the Java library, you can use the `asSet` and `asMap` methods in a trie or associative trie, respectively, to use the trie as a backing trie for a Java Collections Set or Map.

When handling large numbers of addresses or CIDR prefix blocks, it can be much more efficient to use the trie data structure for common operations on those addresses and blocks, the trie constructed in linear time proportional to the number of addresses, and then offering constant time containment and retrieval operations on all of the contained addresses or subnets at once.

Here is an IPv6 trie constructed and then converted to a string, first in Java:

```java
<T extends AddressTrie<A>, A extends Address> T populateTree(
                        T trie, String addrStrs[], Function<String, A> creator) {
        for(String addrStr : addrStrs) {
                A addr = creator.apply(addrStr);
                trie.add(addr);
        }
        return trie;
}

String ipv6Addresses[] = {
        "1::ffff:2:3:5",
        "1::ffff:2:3:4",
        "1::ffff:2:3:6",
        "1::ffff:2:3:12",
        "1::ffff:aa:3:4",
        "1::ff:aa:3:4",
        "1::ff:aa:3:12",
        "bb::ffff:2:3:6",
        "bb::ffff:2:3:12",
        "bb::ffff:2:3:22",
        "bb::ffff:2:3:32",
        "bb::ffff:2:3:42",
        "bb::ffff:2:3:43",
};
IPv6AddressTrie ipv6Trie = populateTree(
        new IPv6AddressTrie(),
        ipv6Addresses,
        str -> new IPAddressString(str).getAddress().toIPv6());
System.out.println(ipv6Trie);
```

and now here is the code in Go:

```go
ipv6Addresses := []string{
  "1::ffff:2:3:5",
  "1::ffff:2:3:4",
  "1::ffff:2:3:6",
  "1::ffff:2:3:12",
  "1::ffff:aa:3:4",
  "1::ff:aa:3:4",
  "1::ff:aa:3:12",
  "bb::ffff:2:3:6",
  "bb::ffff:2:3:12",
  "bb::ffff:2:3:22",
  "bb::ffff:2:3:32",
  "bb::ffff:2:3:42",
  "bb::ffff:2:3:43",
}
ipv6Trie := populateTree(ipv6Addresses)
fmt.Println(ipv6Trie)
```

```go
func populateTree(addrStrs []string) ipaddr.Trie[*ipaddr.IPAddress] {
        trie := ipaddr.Trie[*ipaddr.IPAddress]{}
        for _, addrStr := range addrStrs {
        addr := ipaddr.NewIPAddressString(addrStr).GetAddress()
        trie.Add(addr)
        }
        return trie
}
```

Output from both the Java and Go code:

```
○ ::/0 (13)
└─○ ::/8 (13)
  ├─○ 1::/64 (7)
  │ ├─○ 1::ff:aa:3:0/123 (2)
  │ │ ├─● 1::ff:aa:3:4 (1)
  │ │ └─● 1::ff:aa:3:12 (1)
  │ └─○ 1::ffff:0:0:0/88 (5)
  │   ├─○ 1::ffff:2:3:0/123 (4)
  │   │ ├─○ 1::ffff:2:3:4/126 (3)
  │   │ │ ├─○ 1::ffff:2:3:4/127 (2)
  │   │ │ │ ├─● 1::ffff:2:3:4 (1)
  │   │ │ │ └─● 1::ffff:2:3:5 (1)
  │   │ │ └─● 1::ffff:2:3:6 (1)
  │   │ └─● 1::ffff:2:3:12 (1)
  │   └─● 1::ffff:aa:3:4 (1)
  └─○ bb::ffff:2:3:0/121 (6)
    ├─○ bb::ffff:2:3:0/122 (4)
    │ ├─○ bb::ffff:2:3:0/123 (2)
    │ │ ├─● bb::ffff:2:3:6 (1)
    │ │ └─● bb::ffff:2:3:12 (1)
    │ └─○ bb::ffff:2:3:20/123 (2)
    │   ├─● bb::ffff:2:3:22 (1)
    │   └─● bb::ffff:2:3:32 (1)
    └─○ bb::ffff:2:3:42/127 (2)
      ├─● bb::ffff:2:3:42 (1)
      └─● bb::ffff:2:3:43 (1)
```

An IPv4 trie constructed and then converted to a string using the same polymorphic `populateTree` method, first in Java:

```java
String ipv4Addresses[] = {
        "1.2.3.4",
        "1.2.3.5",
        "1.2.3.6",
        "1.2.3.3",
        "1.2.3.255",
        "2.2.3.5",
```

```
        "2.2.3.128",
        "2.2.3.0/24",
        "2.2.4.0/24",
        "2.2.7.0/24",
        "2.2.4.3",
};
IPv4AddressTrie ipv4Trie = populateTree(
                new IPv4AddressTrie(),
                ipv4Addresses,
                str -> new IPAddressString(str).getAddress().toIPv4());
System.out.println(ipv4Trie);
```

and also in Go:

```
ipv4Addresses := []string{
        "1.2.3.4",
        "1.2.3.5",
        "1.2.3.6",
        "1.2.3.3",
        "1.2.3.255",
        "2.2.3.5",
        "2.2.3.128",
        "2.2.3.0/24",
        "2.2.4.0/24",
        "2.2.7.0/24",
        "2.2.4.3",
}
ipv4Trie := populateTree(ipv4Addresses)
fmt.Println(ipv4Trie)
```

Output from both the Java and Go code:

```
○ 0.0.0.0/0 (11)
└─○ 0.0.0.0/6 (11)
  ├─○ 1.2.3.0/24 (5)
  │ ├─○ 1.2.3.0/29 (4)
  │ │ ├─● 1.2.3.3 (1)
  │ │ └─○ 1.2.3.4/30 (3)
  │ │   ├─○ 1.2.3.4/31 (2)
  │ │   │ ├─● 1.2.3.4 (1)
  │ │   │ └─● 1.2.3.5 (1)
  │ │   └─● 1.2.3.6 (1)
  │ └─● 1.2.3.255 (1)
  └─○ 2.2.0.0/21 (6)
    ├─● 2.2.3.0/24 (3)
    │ ├─● 2.2.3.5 (1)
    │ └─● 2.2.3.128 (1)
    └─○ 2.2.4.0/22 (3)
      ├─● 2.2.4.0/24 (2)
```

```
│ └─● 2.2.4.3 (1)
└─● 2.2.7.0/24 (1)
```

A more compact non-binary representation of the same IPv4 trie, both Java and Go, respectively:

```java
System.out.println(ipv4Trie.toAddedNodesTreeString());
```

```go
fmt.Println(ipv4Trie.AddedNodesTreeString())
```

Output from both the Java and Go code:

```
○ 0.0.0.0/0
├─● 1.2.3.3
├─● 1.2.3.4
├─● 1.2.3.5
├─● 1.2.3.6
├─● 1.2.3.255
├─● 2.2.3.0/24
│ ├─● 2.2.3.5
│ └─● 2.2.3.128
├─● 2.2.4.0/24
│ └─● 2.2.4.3
└─● 2.2.7.0/24
```

Partitioning Subnets

An address trie stores individual addresses or CIDR prefix blocks subnets. There are IPAddress instances that cannot be added to a trie as-is, subnets that are not CIDR prefix blocks. Such subnets can be subdivided or partitioned to addresses or CIDR prefix blocks in various ways.

The `Partition` type encapsulates a partition of a subnet. It also provides a couple of methods that subdivide any subnet into individual addresses or prefix block subnets, which can then be inserted into a trie. Much like an iterator, a partition can be used only once. Simply create another whenever necessary.

The two partition methods provided partition differently. `partitionWithSingleBlockSize` finds a maximal prefix block size and then iterates through a series of prefix blocks of that size. `partitionWithSpanningBlocks` uses any number of different prefix block sizes, which frequently results in a smaller total number of blocks. In fact, it results in the minimal number of prefix blocks, and is used elsewhere in the library to span address ranges with prefix blocks, and is also used by containment tries when converting to prefix blocks.

Here we partition an IPv4 subnet with `partitionWithSingleBlockSize` and then check for the partitioned elements in the trie, first in Java:

```
String addrs = "1.2.1-7.*";
IPv4AddressTrie trie = new IPv4AddressTrie();
IPv4Address subnet = new IPAddressString(addrs).getAddress().toIPv4();
Partition.partitionWithSingleBlockSize(subnet).predicateForEach(trie::add);
boolean foundThemAll = Partition.partitionWithSingleBlockSize(subnet).predicateForEach(trie::c
System.out.println("all inserted: " + foundThemAll);
System.out.println(trie);
```

Here is the equivalent Go code:

```
addrs := "1.2.1-7.*"
trie := ipaddr.Trie[*ipaddr.IPAddress]{}
subnet := ipaddr.NewIPAddressString(addrs).GetAddress()
ipaddr.PartitionWithSingleBlockSize(subnet).PredicateForEach(trie.Add)
foundThemAll := ipaddr.PartitionWithSingleBlockSize(subnet).PredicateForEach(trie.Contains)
fmt.Println("all inserted:", foundThemAll)
fmt.Println(trie)
```

Output from both the Java and Go code:

```
all inserted: true

○ 0.0.0.0/0 (7)
└─○ 1.2.0.0/21 (7)
  ├─○ 1.2.0.0/22 (3)
  │ ├─● 1.2.1.0/24 (1)
  │ └─○ 1.2.2.0/23 (2)
  │   ├─● 1.2.2.0/24 (1)
  │   └─● 1.2.3.0/24 (1)
  └─○ 1.2.4.0/22 (4)
    ├─○ 1.2.4.0/23 (2)
    │ ├─● 1.2.4.0/24 (1)
    │ └─● 1.2.5.0/24 (1)
    └─○ 1.2.6.0/23 (2)
      ├─● 1.2.6.0/24 (1)
      └─● 1.2.7.0/24 (1)
```

Following that code, we try the other partition method `partitionWithSpanningBlocks` on the same subnet, with Java code:

```java
trie = new IPv4AddressTrie();
Partition.partitionWithSpanningBlocks(subnet).predicateForEach(trie::add);
foundThemAll = Partition.partitionWithSpanningBlocks(subnet).predicateForEach(trie::contains);
System.out.println("all inserted: " + foundThemAll);
System.out.println(trie);
```

Here we try the other partition method `PartitionWithSpanningBlocks` on the same subnet, with Go code:

```go
trie = ipaddr.Trie[*ipaddr.IPAddress]{}
ipaddr.PartitionWithSpanningBlocks(subnet).PredicateForEach(trie.Add)
foundThemAll = ipaddr.PartitionWithSpanningBlocks(subnet).PredicateForEach(trie.Contains)
fmt.Println("all inserted:", foundThemAll)
fmt.Println(trie)
```

Output from both the Java and Go code:

```
all inserted: true

○ 0.0.0.0/0 (3)
└─○ 1.2.0.0/21 (3)
  ├─○ 1.2.0.0/22 (2)
  │ ├─● 1.2.1.0/24 (1)
  │ └─● 1.2.2.0/23 (1)
  └─● 1.2.4.0/22 (1)
```

The two tries illustrate how the two partitions differ.

Dual Trie Types

It is the individual bit values in each address that determines its location in a trie. For this reason, the bits across all keys of a trie must be consistent. Therefore, you cannot have addresses with different bit lengths in the same trie.

Some trie types are type-safe to restrict keys to a specific address type or version, like IPv4. However, the library does provide the `IPAddressTrie` and `IPAddressAssociativeTrie` types which allow you to insert either IPv4 or IPv6 addresses into the trie. Still, you can have only one address version or the other in the trie at any time, for the reason described above. The version of the first address added to an empty `IPAddressTrie` or `IPAddressAssociativeTrie` trie determines what version all additional addresses must be, at least until the trie is empty again, at which time it can once again accept an address of either version.

If you do wish to use the trie data structure for both IPv4 and IPv6 addresses at the same time, the library provides the “Dual” trie types, `DualIPv4v6Tries` and `DualIPv4v6AssociativeTries`. These types actually encapsulate a pair of tries, one for the IPv4 keys, another for the IPv6 keys. Otherwise, they have all the

same operations. This provides an additional layer of polymorphism for applications that may benefit from remaining IP-version agnostic.

IP Address Collections

A collection of IP addresses is more expansive and encompassing than a single subnet or sequential range, and can represent any arbitrary set of IP addresses.

While it is possible to maintain collections of individual IP addresses using the Java Collections types, storing addresses individually is not efficient in terms of performance or memory.

Instances of `IPAddressCollection` are more efficient, backed by prefix blocks or sequential ranges, storing addresses into the minimal number of prefix blocks or sequential ranges possible.

The library provides two options for maintaining space-efficient and performance-optimized collections of IP addresses, `IPAddressSeqRangeList` and `IPAddressContainmentTrie`.

`IPAddressSeqRangeList` is backed by an array of sequential ranges. `IPAddressContainmentTrie` is backed by a trie of CIDR prefix blocks. Both offer binary search for containment queries. Whether one is better than the other may depend on the data set or the underlying processor, or whether you may need additional operations that are specific to one collection or the other. In particular, `IPAddressSeqRangeList` is likely to offer better cache coherency in CPU processors, which may result in better overall performance in searching. However, when dealing with CIDR prefix blocks as your input or output, it may be more efficient to go with IPAddressContainmentTrie since the elements in the collection are also stored as prefix blocks, requiring little to no conversion when adding or removing.

Both collection options change shape internally as addresses are added and removed, and for this reason they do not allow direct access to the backing list or the backing trie. The backing data structures change shape so that they always contain the minimal number of sequential ranges or prefix blocks to represent the collection of addresses.

When adding to either collection option, whether adding an IP address, subnet, or sequential range, the argument is converted to the data types used by the collection's backing data structure. So, in particular, unlike address tries, which require that callers convert to prefix blocks or individual address first, the containment trie does this conversion for you, allowing an argument to be an address, subnet, or range of any shape.

Containment tries also differ from regular address tries in that the elements of containment tries are the individual addresses. With address tries, the elements are both prefix blocks and individual addresses. In an address trie, an individual address might be added individually to the trie, while it can also exist in the trie as part of an added subnet, at the same time. With a containment trie, that is not the case. An address can be located in just one block in the backing trie data structure. However, the location may change. An address

element that is part of the collection might not remain in the same location in the backing trie as the trie changes shape. Regardless, there will always be exactly one block or address in the backing trie holding that address element.

Another difference between address tries and containment tries is the string representation. When producing a string of an address trie, the counts associated with each node are the added node counts, because the elements are the added nodes, which represent either prefix blocks or individual addresses. In a containment trie the counts associated with each node are the address counts, because the elements are the individual addresses.

To illustrate, here we add the same set of addresses and blocks to a trie, a containment trie collection, and a range list collection. In the trie, the blocks and addresses are the elements, and the bracketed roll-up counts shown with each node show the count of those elements, showing that we added 11 blocks and addresses. With the containment trie, the elements are the individual addresses of each added subnet and address, so the original blocks and addresses are not present in the trie as the trie has changed shape to have the minimal number of nodes. Also, the roll-up shows the count of individual addresses added, which totals 4608. The list also changes shape as the subnets and addresses are added, so as to contain the minimal number of sequential range lists.

```
String addrStrs[] = {
                "10.172.80.0/21", "10.172.88.0/24", "10.172.89.0/24", "10.172.88.0",
                "10.172.88.3", "10.172.88.5", "10.172.96.0/21", "10.172.89.1",
                "10.172.81.0/24", "10.172.82.0/24", "10.172.83.0/24"};
IPAddressTrie trie = new IPAddressTrie();
IPAddressContainmentTrie trieCollection = new IPAddressContainmentTrie();
IPAddressSeqRangeList listCollection = new IPAddressSeqRangeList();
for(String str : addrStrs) {
        IPAddress addr = new IPAddressString(str).getAddress();
        trie.add(addr);
        trieCollection.add(addr);
        listCollection.add(addr);
}
System.out.println("trie with " + trie.size() + " elements: " + trie);
System.out.println("containment trie collection with " +
                trieCollection.getCount() + " elements: " + trieCollection);
System.out.println("sequential range list collection with " +
                listCollection.getCount() + " elements:\n" + listCollection);
```

Output:

```
trie with 11 elements:
○ 0.0.0.0/0 (11)
└─○ 10.172.64.0/18 (11)
  ├─○ 10.172.80.0/20 (10)
  │ ├─● 10.172.80.0/21 (4)
  │ │ └─○ 10.172.80.0/22 (3)
```

```
  │ │   ├─● 10.172.81.0/24 (1)
  │ │   └─○ 10.172.82.0/23 (2)
  │ │     ├─● 10.172.82.0/24 (1)
  │ │     └─● 10.172.83.0/24 (1)
  │ └─○ 10.172.88.0/23 (6)
  │   ├─● 10.172.88.0/24 (4)
  │   │ └─○ 10.172.88.0/29 (3)
  │   │   ├─○ 10.172.88.0/30 (2)
  │   │   │ ├─● 10.172.88.0 (1)
  │   │   │ └─● 10.172.88.3 (1)
  │   │   └─● 10.172.88.5 (1)
  │   └─● 10.172.89.0/24 (2)
  │     └─● 10.172.89.1 (1)
  └─● 10.172.96.0/21 (1)

containment trie collection with 4608 elements:
○ 0.0.0.0/0 (4608)
└─○ 10.172.64.0/18 (4608)
  ├─○ 10.172.80.0/20 (2560)
  │ ├─● 10.172.80.0/21 (2048)
  │ └─● 10.172.88.0/23 (512)
  └─● 10.172.96.0/21 (2048)

sequential range list collection with 4608 elements:
[10.172.80.0 -> 10.172.89.255, 10.172.96.0 -> 10.172.103.255]
```

Both collection options `IPAddressSeqRangeList` and `IPAddressContainmentTrie` implement the interface `IPAddressCollection`, sharing the same collection operations.

`IPAddressSeqRangeList` offers a number of additional operations. It has operations offering quick retrieval of addresses by index. It has additional set operations such as `intersect`. In fact, you can do set arithmetic with `IPAddressSeqRangeList` quite easily, and you can choose to do so either by operating on an instance in place, or by producing new lists with each operation. For instance, see the example illustrating De Morgan's laws of set theory.

Equality of an `IPAddressCollection` instance with another instance of `IPAddressCollection` is determined by the contents of the collections. They are equal if the collections contain the same set of individual addresses.

IP Address Aggregations

As you an see in this document, there are a number of ways in this library to represent a multitude of addresses with a single object, whether it be a subnet, a sequential range, or a collection. There are also options allowing you to convert from one to the other, or to compare.

The IPAddressAggregation interface represents all types that can represent a multitude of individual addresses. This includes subnets (`IPAddress`), sequential ranges (`IPAddressSeqRange`), and address

collections (`IPAddressSeqRangeList` and `IPAddressContainmentTrie`).

IP Address Operations

Here we summarize the operations on IP addresses, subnets, sequential ranges, and collections. Many of these operations are also available on other address items, such as sections and segments.

We start with the more general operations, those that do not involve prefixes, prefix blocks, or segment blocks, followed by operations involving prefixes and prefix blocks, followed the operations involving segment blocks. Segment block subnets are those subnets which have value ranges by segment. Prefix blocks are subnets that have a value range corresponding to a specified prefix length, the subnet containing the full block of addresses according to that prefix.

General Operations on Subnets, Addresses and Sequential Ranges

Many of these operations are available on address sections as well. You can obtain an address section by either constructing one directly or by getting the address section from a subnet or address.

Java	Go	Description
equals	Equal	Returns whether two address items or sequential ranges include the exact same individual values
contains	Contains, ContainsRange	Returns whether an address item or sequential range includes all the individual values of another address item or sequential range
isMultiple	IsMultiple	Returns whether an address item or sequential range includes multiple values within its range.
getCount	GetCount	Returns the number of individual values contained with the address item or sequential range, the number of values within its range of values.
getLower	GetLower	Returns the lowest single-valued individual address item contained within an address item or sequential range.
getUpper	GetUpper	Returns the highest single-valued individual address item contained within an address item or sequential range

Java	Go	Description
includesMax	IncludesMax	Returns whether the range of values in the address item or sequential range includes the largest possible individual value.
includesZero	IncludesZero	Returns whether the range of values in the address item or sequential range includes the smallest possible individual value, namely the value of zero.
isMax	IsMax	Returns whether the the address item or sequential range includes just a single value, which is the largest possible individual value.
isZero	IsZero	Returns whether the the address item or sequential range includes just a single value, which is the smallest possible individual value, namely the value of zero.
isFullRange	IsFullRange	Returns whether the range of values in the address item or sequential range includes all possible values, from zero to the max value.
isSequential	IsSequential	Returns whether the range of values within the address item are a sequence of consecutive values. In other words, it is sequential if the number of disjoint value ranges in the address item is one. This really only applies to address items that are not sequential ranges since, by definition, a sequential range is always sequential, representing the sequence of addresses between a pair of addresses.
iterator, spliterator, stream	Iterator	Traverses through the individual address items (or sections thereof) comprising the range of values within the original address item or sequential range. Use `getCount` / `GetCount` to get the traversed count.
subtract, subtractIntoList	Subtract	Computes the difference, the set of addresses in the receiver address, subnet, or sequential range, but not in the argument address, subnet, or sequential range. Returns an array of subnets or or sequential ranges containing the result. The `IntoList` variant efficiently produces a sorted list.
intersect	Intersect	Computes the conjunction with the given address, subnet or sequential range arguments. The conjunction is the set of addresses in both the receiver and all

Java	Go	Description
		arguments. Returns the address, subnet or sequential range representing that set of addresses.
complement, complementIntoList		Computes the complement of the address, subnet or sequential range. Returns the address, subnet or sequential range representing those addresses that were not in the original receiver address, subnet or sequential range. The `IntoList` variant efficiently produces a sorted list.
overlaps	Overlaps	Returns whether the sequential range overlaps with the given address, subnet, or sequential range.
intoSequentialRangeList		Constructs a sequential range list with the same set of individual addresses as the address, subnet, or sequential range

Other General Operations on Subnets and Addresses

Many of these operations are available on address sections as well. You can obtain an address section by either constructing one directly or by getting the address section from a subnet or address.

Java	Go	Description
increment, decrement	Increment	If the incremented address item is a subnet, provide individual address that is the given increment into th sequence of individual addresses within the subnet r An increment exceeding the subnet count being sim added to the final address in the subnet. If the addre individual address, simply adds the given increment address value to produce a new address. The increm value can be a positive or negative integer.
incrementBoundary	IncrementBoundary	Returns the address that is the given increment from the range boundaries of the subnet or address, with increments added to the upper bound of the range, a negative increments being added to the lower bound increment of zero returns the original. If the address individual address, simply adds the given increment address value to produce a new address.

Java	Go	Description
enumerate	Enumerate	Returns the index in the subnet of the given individu address. If the given address is above or below the s gives the distance of the given address to the neare subnet boundary. Note that this gives the distance b two individual addresses. This operation is the invers increment operation, so that `subnet.enumerate(subnet.increment(incrementVa` `incrementValue` and `subnet.increment(subnet.enumerate(individualA` `= individualAddress`
reverseBits, reverseBytes, reverseBytesPerSegment, reverseSegments	ReverseBits, ReverseBytes, ReverseSegments	Reverses the bits of segments, bytes, or the entire a or subnet. Each individual value is reversed and inclu the result. Note that some subnets cannot have bits reversed due to a reversed segment not being expre as a single range of segment values. Reverse operat be useful for handling endianness (network byte ord sometimes requires bytes be reversed), or DNS look
toIPv4, toIPv6	ToIPv4, ToIPv6	Provides the same address represented with the mo specific type, so that IP-version specific method call made. If the address was originally constructed as th specific type, then an instance of that type is returne address itself remains the same.
segmentsIterator, segmentsSpliterator, segmentsStream		Traverses through all address items, similar to `iterator` / `spliterator` / `stream` , but using only se arrays. Use `getCount` to get the count.
coverWithSequentialRange	ToSequentialRange	Returns the associated sequential range ranging fro lowest to highest value of the address or subnet. You use the `isSequential` / `IsSequential` method of t address to know if the resulting sequential range rep the same set of addresses as the original address or
spanWithRange	SpanWithRange	Returns a sequential range that spans from the subn the given subnet. The resulting range will include all addresses in both subnets as well as all addresses in between.
matchOrdered,	MatchOrdered,	Returns whether two arrays of addresses contain the

Other General Operations on Sequential Ranges

Java	Go	Description
extend	Extend	Extend extends the sequential range to include all address in the given sequential range, as well as all address in-between the two sequential ranges.
join, jointIntoList	Join, JoinTo	Given a list of address ranges, merges them into the minimal list of address ranges. The `IntoList` variant efficiently produces a sorted list.
enumerate	Enumerate	Returns the index in the range of the given individual address. If the given address is above or below the range, gives the distance of the given address to the nearest range boundary.
get(addressIndex)		Returns the individual address at the given index in the sequential range
lowerFromSplit, upperFromSplit, split		Splits a range in two and returns the lower or upper sides, or both
toIPv4, toIPv6	ToIPv4, ToIPv6	Provides the same sequential range instance with the more specific type, so that IP-version specific method calls can be made.

Operations Involving Prefixes or Prefix Blocks

The following methods on addresses, subnets, and sequential ranges allow you to query whether something is a prefix block, change prefix lengths, mask addresses, or other prefix-related operations, which are particularly integral to CIDR addressing and routing.

Java	Go	Description
prefixEquals	PrefixEqual	Returns whether the prefix of the address item matches the same bits in the given address or subnet.
prefixContains	PrefixContains	Returns whether the prefix of the address item contains all the values

Java	Go	Description
		of the same bits in the given address or subnet.
isPrefixBlock	IsPrefixBlock	Returns whether the address item has a prefix length and contains the prefix block for that prefix. A prefix block subnet, such as the /64 block a:b:c:d::/64 or the /16 block 1.2.0.0/16, is a subnet or collection of address sections that contains all the addresses or address sections with the same prefix, with a prefix length identifying the prefix.
isSinglePrefixBlock	IsSinglePrefixBlock	Returns whether the address item has a prefix length, it has a single prefix of that prefix length, and contains the prefix block for that prefix.
containsSinglePrefixBlock	ContainsSinglePrefixBlock	Returns whether the subnet or sequential range contains the prefix block for the given prefix length, regardless of the assigned prefix length.
containsPrefixBlock	ContainsPrefixBlock	Returns whether the subnet or sequential range contains the prefix block for the given prefix length and has a single prefix of that prefix length, regardless of the assigned prefix length.
toPrefixBlock	ToPrefixBlock, ToPrefixBlockLen	Returns the subnet comprising the entire prefix block (all addresses with the same prefix), with either the existing or a given prefix length.
getMinPrefixLengthForBlock	GetMinPrefixLenForBlock	Returns the smallest prefix length for which the range matches the block of addresses for that prefix.
assignMinPrefixForBlock	AssignMinPrefixForBlock	Converts to an equivalent subnet (or section thereof) with the smallest

Java	Go	Description
		prefix length, so that the result is a prefix block for that prefix length. The result will span the same range of values as the original.
getPrefixLengthForSingleBlock	GetPrefixLenForSingleBlock	Returns a prefix length for which the range matches exactly the prefix block for a single-valued prefix, if such a prefix length exists.
assignPrefixForSingleBlock	AssignPrefixForSingleBlock	Provides the equivalent subnet or address with a prefix length, the prefix length being the prefix length that makes the returned subnet a single prefix block. Such a subnet or address might not exist. The resulting subnet or address, if it exists, will span the same range of values as the original, and will thus remain equal to the original.
getBlockMaskPrefixLength	GetBlockMaskPrefixLen	Returns the prefix length of a network or host mask, if the address is a mask.
withoutPrefixLength	WithoutPrefixLen	Convert to the same but with no prefix length
setPrefixLength	SetPrefixLen, SetPrefixLenZeroed	Set the prefix length to the indicated value in the result. If there was an existing prefix length, then the “zeroed” variants control whether the bits to change sides of the prefix are changed to zeros, or whether the bits retain their values from when they were on the other side of the prefix boundary. Other bits retain their value. *Java only*: another method variant determines if the result can be re-interpreted as a prefix block when the host is all zeros, like when parsing strings with

Java	Go	Description
		zero hosts or creating addresses with zero hosts.
adjustPrefixLength	AdjustPrefixLen, AdjustPrefixLenZeroed	Adjust the prefix length by the indicated value to a new prefix length in the result. The “zeroed” variants control whether the bits to change sides of the prefix are changed to zeros, or whether the bits retain their values from when they were on the other side of the prefix boundary. Other bits retain their value. *Java only*: another method variant determines if the result can be re-interpreted as a prefix block when the host is all zeros, like when parsing strings with zero hosts or creating addresses with zero hosts.
adjustPrefixBySegment		Adjust the prefix length to the next segment boundary in the result. The “zeroed” variants control whether the bits to change sides of the prefix are changed to zeros, or whether the bits retain their values from when they were on the other side of the prefix boundary. Other bits retain their value.
mask, maskNetwork	Mask	Applies a mask to a subnet or address, the result being the bitwise conjunction. For subnets, the mask is applied to all individual addresses to produce a single result, if possible. *Java only*: the “network” variant allows you to mask just the network and apply a prefix length.
bitwiseOr, bitwiseOrNetwork	BitwiseOr	Produces the bitwise disjunction of the original with the given mask. For subnets, the mask is applied to all

Java	Go	Description
		individual addresses to produce a single result, if possible. *Java only*: the “network” variant allows you to mask just the network and apply a prefix length.
matchesWithMask	MatchesWithMask	Applies a mask to an address item and then compares the result with another address item, returning true if they match, false otherwise.
includesZeroHost	IncludesZeroHost IncludesZeroHostLen	Returns whether the host part of the address item, the bits following the prefix, includes the value of zero within its range.
includesMaxHost	IncludesMaxHost IncludesMaxHostLen	Returns whether the host part of the address item, the bits following the prefix, includes the maximum possible value within its range, the value in which all host bits are ones.
isZeroHost	IsZeroHost, IsZeroHostLen	Returns whether the host part of the address item, the sequence of bits following the prefix, is single-valued, that value being zero.
	IsMaxHost, IsMaxHostLen	Returns whether the host part of the address item, the sequence of bits following the prefix, is single-valued and is the maximum possible value within its range, the value in which all host bits are ones.
toZeroHost	ToZeroHost, ToZeroHostLen	Produces the address item with prefix having the same range of values as the original and with the host, the sequence of bits beyond the prefix, having the value of zero.
toZeroNetwork	ToZeroNetwork	Produces the address item with the prefix, the bits within the prefix length, having the value of zero, and

Java	Go	Description
		the host the same range of values as the original.
toMaxHost	ToMaxHost, ToMaxHostLen	Produces the address item with the same prefix as the original and with a host, the sequence of bits beyond the prefix, having the maximum possible value, the value where all host bits are ones.
prefixBlockIterator, prefixBlockSpliterator, prefixBlockStream	PrefixBlockIterator	If the subnet (or section thereof) has a prefix length, then this traverses through the prefix blocks subnets for that prefix length, with each traversed item being a prefix block. If no prefix length, then it traverses through all individual addresses (or section thereof). Use `getPrefixCount / GetPrefixCount` for the traversed count. For sequential range variants, the prefix length for the traversal is supplied.
prefixIterator, prefixSpliterator, prefixStream	PrefixIterator	If the subnet (or section thereof) has a prefix length, then traverses through the prefixes, with each traversed item including all those items from the original which have the same prefix. All except possibly the boundary iterations (first and last) will be a prefix block. If no prefix length, then it traverses through all individual addresses (or sections thereof). Use `getPrefixCount / GetPrefixCount` for the traversed count. For sequential range variants, the prefix length for the traversal is supplied.
nonZeroHostIterator		Traverses through the individual address items (or sections thereof) comprising the range of values

Java	Go	Description
		within the original address item, but skipping those values with a zero host, if the original has a prefix length. Values with a zero host typically represent the entire block with the same prefix, rather than an individual address. Use `getNonZeroHostCount` to get the traversed count.
getPrefixCount	GetPrefixCount, GetPrefixCountLen	Returns the number of prefixes contained with the address item or sequential range, for either the existing prefix (which is the entire address item if no existing prefix length) or the given prefix length. For sequential range variants, the prefix length is given.
spanWithPrefixBlocks	SpanWithPrefixBlocks, SpanWithPrefixBlocksTo	Given a pair of addresses or subnets (or sections thereof), or a sequential range, finds the minimal list of prefix block subnets that span all addresses within.
mergeToPrefixBlocks	MergeToPrefixBlocks	Given a list of addresses or subnets (or sections thereof), merges them into the minimal list of prefix blocks.
coverWithPrefixBlock	CoverWithPrefixBlock, CoverWithPrefixBlockTo	Given a pair of addresses or subnets (or sections thereof), or a sequential range, returns the minimal-size prefix block that includes them both

Operations involving Segment Blocks

The following operations involve segment blocks. Note that a sequential block is a segment block segment that is sequential, such that for any two addresses in the range, all addresses in-between are also in the range. To be sequential, any multi-valued segment must be followed only by segments comprising all segment values.

Java	Go	Description
blockIterator, blockSpliterator, blockStream	BlockIterator	Traverses through the segment values of the higher segments up until a given segment index, with each traversed item including all those items from the original which have the same initial single-valued segments. If the segment index is 0, produces just a single value, the original address. If the segment index matches the segment count, traverses through all individual addresses (or sections thereof). Use `getBlockCount` / `GetBlockCount` to get the traversed count.
getBlockCount	GetBlockCount	Returns the count of distinct individual values in the given number of initial (more significant) segments.
sequentialBlockIterator, sequentialBlockSpliterator, sequentialBlockStream	SequentialBlockIterator	Traverses through the segment values of the higher segments up until a given segment index, with each traversed item including all those from the original which have the same initial single-valued segments. The segment index is chosen to be the largest segment index for which all iterated blocks will be sequential. Use `getSequentialBlockCount` / `GetSequentialBlockCount` to get the traversed count.
getSequentialBlockCount	GetSequentialBlockCount	Returns the count of maximal-length sequential value ranges in the address item. In other words, it is the number of disjoint value ranges in the address item. If `isSequential` / `IsSequential` returns true, the count will be one.
spanWithSequentialBlocks	SpanWithSequentialBlocks, SpanWithSequentialBlocksTo	Given a pair of addresses or subnets (or sections thereof), or a sequential range, finds the minimal list of

Java	Go	Description
		sequential block subnets that span all addresses within.
mergeToSequentialBlocks	MergeToSequentialBlocks	Given a list of addresses or subnets (or sections thereof), merges them into the minimal list of sequential block subnets
append, prepend, replace	Replace, ReplaceLen	Adds or replaces segments in the subnet, address, or the section of a subnet or address. With subnets and addresses you must always maintain the correct number of segments, so only the replace operation is provided.

Example of Mask and Prefix Length Operations

The following code demonstrates how to get the lowest address in a prefixed subnet using any one of three methods: getting the lowest address in the subnet, masking, or converting the host to zero.

```
String addr = "1.2.3.4";
int prefixLength = 16;

IPAddress address = new IPAddressString(addr).getAddress();
IPAddress mask = address.getNetwork().getNetworkMask(prefixLength, false);
System.out.println("mask " + mask);


// create the prefix block subnet

IPAddress subnet = address.setPrefixLength(prefixLength).toPrefixBlock();
System.out.println("subnet " + subnet + " no prefix is " +
                subnet.withoutPrefixLength());


// mask

IPAddress maskedSubnet = subnet.mask(mask);
System.out.println("subnet " + subnet + " masked is " + maskedSubnet);
IPAddress maskedAddress = address.mask(mask);
System.out.println("address " + address + " masked is " + maskedAddress);
System.out.println("masked address " + maskedAddress +
                " equals masked subnet " + maskedSubnet + " is " +
                maskedAddress.equals(maskedSubnet));
```

```java
// get the lower address

IPAddress lowestAddrInSubnet = subnet.getLower();
System.out.println("lowest in subnet is " + lowestAddrInSubnet);
System.out.println("lowest in subnet no prefix is " +
                lowestAddrInSubnet.withoutPrefixLength());
System.out.println("masked address " + maskedAddress +
                " equals lowest address in subnet " + lowestAddrInSubnet +  " is " +
                maskedAddress.equals(lowestAddrInSubnet));

// get the zero host

IPAddress zeroHost = subnet.toZeroHost();
System.out.println("zero host is " + zeroHost);
System.out.println("zero host no prefix is " +
                zeroHost.withoutPrefixLength());
System.out.println("masked address " + maskedAddress +
                " equals zero host " + zeroHost +  " is " +
                maskedAddress.equals(zeroHost));
```

Here is the equivalent Go code:

```go
addr := "1.2.3.4"
prefixLength := 16

address := ipaddr.NewIPAddressString(addr).GetAddress()
mask := address.GetNetwork().GetNetworkMask(prefixLength)
fmt.Println("mask", mask)

// create the prefix block subnet

subnet := address.SetPrefixLen(prefixLength).ToPrefixBlock()
fmt.Println("subnet", subnet, "no prefix is", subnet.WithoutPrefixLen())

// mask

maskedSubnet, _ := subnet.Mask(mask)
maskedSubnet = maskedSubnet.WithoutPrefixLen()
fmt.Println("subnet", subnet, "masked is", maskedSubnet)
maskedAddress, _ := address.Mask(mask)
fmt.Println("address", address, "masked is", maskedAddress)
fmt.Println("masked address", maskedAddress,
        "equals masked subnet", maskedSubnet,
        "is", maskedAddress.Equal(maskedSubnet))

// get the lower address

lowestAddrInSubnet := subnet.GetLower()
fmt.Println("lowest in subnet is", lowestAddrInSubnet)
fmt.Println("lowest in subnet no prefix is",
```

```
    lowestAddrInSubnet.WithoutPrefixLen())
  fmt.Println("masked address", maskedAddress,
          "equals lowest address in subnet", lowestAddrInSubnet,
          "is", maskedAddress.Equal(lowestAddrInSubnet))

  // get the zero host

  zeroHost, _ := subnet.ToZeroHost()
  fmt.Println("zero host is", zeroHost)
  fmt.Println("zero host no prefix is", zeroHost.WithoutPrefixLen())
  fmt.Println("masked address", maskedAddress,
          "equals zero host", zeroHost,
          "is", maskedAddress.Equal(zeroHost))
```

Here is the output from either the Java or Go code:

```
mask 255.255.0.0
subnet 1.2.0.0/16 no prefix is 1.2.*.*
subnet 1.2.0.0/16 masked is 1.2.0.0
address 1.2.3.4 masked is 1.2.0.0
masked address 1.2.0.0 equals masked subnet 1.2.0.0 is true
lowest in subnet is 1.2.0.0/16
lowest in subnet no prefix is 1.2.0.0
masked address 1.2.0.0 equals lowest address in subnet 1.2.0.0/16 is true
zero host is 1.2.0.0/16
zero host no prefix is 1.2.0.0
masked address 1.2.0.0 equals zero host 1.2.0.0/16 is true
```

Polymorphism

The address framework of interfaces, and the hierarchical types allow for substantial polymorphism with all operations.

As an example, imply change the string “1.2.3.4” in the code above to an IPv6 address like “a:ffff:b:c:d::f” and the code works all the same.

Output:

```
mask ffff::
subnet a::/16 no prefix is a:*:*:*:*:*:*:*
subnet a::/16 masked is a::
address a:ffff:b:c:d::f masked is a::
masked address a:: equals masked subnet a:: is true
lowest in subnet is a::/16
lowest in subnet no prefix is a::
masked address a:: equals lowest address in subnet a::/16 is true
```

```
zero host is a::/16
zero host no prefix is a::
masked address a:: equals zero host a::/16 is true
```

Masking Subnets

When applying an operation to a subnet, the operation is applied to every member of the subnet. In such cases, the result must be something representable with sequential segment ranges, otherwise `IncompatibleAddressException` will be thrown in Java, or an `IncompatibleAddressError` returned in Go. In other words, each resulting segment must be representable by a single range of values.

For instance, masking the subnet block of 255 addresses 0.0.0.0/24 with the mask 0.0.0.128 results in the two addresses 0.0.0.0 and 0.0.0.128, which is not a sequential range of values.

Such exceptions or errors will not happen when using standard masking and subnetting techniques typical with IPv4 and IPv6 routing.

Subnetting

Subnetting can be accomplished using various address manipulation methods. Given a prefixed IP address, you can extend the prefix length and insert bits for an extended prefix and new subnet.

```java
int originalPrefix = 18, adjustment = 4;
IPAddress address = new IPAddressString("207.0.64.0").getAddress();
IPAddress subnet1 = address.toPrefixBlock(originalPrefix);
System.out.println(subnet1 + " of size " + subnet1.getCount());
IPAddress subnet2 = subnet1.adjustPrefixLength(adjustment); // extend the prefix length
System.out.println(subnet2);
IPAddress prefixExtension = new IPAddressString("0.0.4.0").getAddress();
IPAddress subnet3 =
                subnet2.bitwiseOrNetwork(prefixExtension,
                  originalPrefix + adjustment); // adjust the extended prefix
System.out.println(subnet3 + " of size " + subnet3.getCount());
```

The equivalent Go code is:

```go
originalPrefix, adjustment := 18, 4
address := ipaddr.NewIPAddressString("207.0.64.0").GetAddress()
subnet1 := address.ToPrefixBlockLen(originalPrefix)
fmt.Println(subnet1, "of size", subnet1.GetCount())
subnet2, _ := subnet1.AdjustPrefixLenZeroed(adjustment) // extend the prefix length
fmt.Println(subnet2)
prefixExtension := ipaddr.NewIPAddressString("0.0.4.0").GetAddress()
```

```
subnet3, _ := subnet2.BitwiseOr(prefixExtension) // adjust the extended prefix
fmt.Println(subnet3, "of size", subnet3.GetCount())
```

Output for both the Java and Go code:

```
207.0.64.0/18 of size 16384
207.0.64.0/22
207.0.68.0/22 of size 1024
```

Here is the same subnetting operation using segment replacement.

```
IPv4Address address = new IPAddressString("207.0.64.0/18").getAddress().toIPv4();
IPv4AddressSection replacementSection =
                new IPAddressString("0.0.68.0/22").getAddress().toIPv4().getSection(2);
IPAddress subnet = new IPv4Address(address.getSection().replace(2, replacementSection));
System.out.println(subnet + " of size " + subnet.getCount());
```

```
address := ipaddr.NewIPAddressString("207.0.64.0/18").GetAddress().ToIPv4()
replacementSection :=
        ipaddr.NewIPAddressString("0.0.68.0/22").GetAddress().ToIPv4().GetTrailingSection(2)
subnet, _ := ipaddr.NewIPv4Address(address.GetSection().Replace(2, replacementSection))
fmt.Println(subnet, "of size", subnet.GetCount())
```

Output for both the Java and Go code:

```
207.0.68.0/22 of size 1024
```

Alternatively, you can use the prefix block iterator to get a list of subnets when adjusting the prefix:

```
IPAddress subnet = new IPAddressString("192.168.0.0/28").getAddress();
IPAddress newSubnets = subnet.setPrefixLength(subnet.getPrefixLength() + 2, false);
System.out.println(newSubnets);

Iterator<? extends IPAddress> iterator = newSubnets.prefixBlockIterator();
if (iterator.hasNext()) {
        System.out.print(iterator.next());
        while (iterator.hasNext()) {
                System.out.print(", ");
                System.out.print(iterator.next());
        }
}
```

```go
subnet := ipaddr.NewIPAddressString("192.168.0.0/28").GetAddress()
newSubnets := subnet.SetPrefixLen(subnet.GetPrefixLen().Len() + 2)
fmt.Println(newSubnets)

iterator := newSubnets.PrefixBlockIterator()
if iterator.HasNext() {
        fmt.Print(iterator.Next())
        for iterator.HasNext() {
                fmt.Print(", ", iterator.Next())
        }
}
```

Output for both the Java and Go code:

```
192.168.0.0-12/30
192.168.0.0/30, 192.168.0.4/30, 192.168.0.8/30, 192.168.0.12/30
```

Another option is to let the library do most of the work for you. A `PrefixBlockAllocator` instance can do CIDR subnetting using a standard variable-length subnetting algorithm.

The Java wiki and Go wiki provide subnetting code examples as well as examples demonstrating how to create or derive CIDR subnets, including a Java example and Go example showing use of the `PrefixBlockAllocator`.

Operations with Tries

On the Java side, tries and associative tries can be converted into sets and maps that are backed by the original trie instances. The set and map types integrate with the types in the Java collections framework. In fact, the `AddressTrieSet` type implements the Java collections framework interfaces `java.util.Collection`, `java.util.NavigableSet`, `java.util.Set`, and `java.util.SortedSet`. The `AddressTrieMap` type implements the Java collections framework interfaces `java.util.Map`, `java.util.NavigableMap`, `java.util.SortedMap`. The map and set types can be used with code that operates on Java collections framework interfaces. The Map.Entry instances from the maps correspond to the AssociativeTrieNode instances from the maps' backing associative tries. In this table, operations *in italics* are Java collections framework operations on the trie-backed sets and maps. Those operations are methods that operate on the backing trie in the set or map instance.

Java	Go	Description
add, addNode, addTrie, *add*, *addAll*	Add, AddNode, AddTrie	Insert one or more nodes into the trie f individual address(es) or CIDR prefix b

Java	Go	Description
addIfNoElementsContaining		Similar to the methods above, but cont as a match. This operation is useful for tries.
put, putNew, putNode, putTrie, *put*, *putAll*	Put, PutNode, PutTrie	Insert one or more nodes into the trie f individual address(es) or CIDR prefix b with the given associated value. If the subnet already exists in the trie, the as any, is replaced.
get, getNode, getAddedNode, contains, containsAll, containsKey	Get, GetNode, GetAddedNode, Contains	Retrieves or checks for the existence c trie whose key matches the given indiv or CIDR prefix block
remove, *remove*, *removeAll*	Remove	Removes from the trie the nodes whos given individual addresses or CIDR pre
ceiling, floor, higher, lower, ceilingAddedNode, floorAddedNode, higherAddedNode, lowerAddedNode, *ceiling*, *floor*, *higher*, *lower*, *ceilingKey*, *floorKey*, *higherKey*, *lowerKey*, *ceilingEntry*, *floorEntry*, *higherEntry*, *lowerEntry*	Ceiling, Floor, Higher, Lower, CeilingAddedNode, FloorAddedNode, HigherAddedNode, LowerAddedNode	Finds the node whose key is closest (a order) to the given individual address c block. `Ceiling` matches the lowest ke equal to the given address or CIDR pre matches the highest key less than or e address or CIDR prefix block. `Higher` lowest key greater than the given addr prefix block. `Lower` matches the highe the given address or CIDR prefix block
containingCeilingAddedNode, containingFloorAddedNode, containingHigherAddedNode, containingLowerAddedNode		Finds the node whose key contains the (according to trie order) to the given ir or CIDR prefix block. It is similar to finc like with the above methods, except th counts as a match, an exact match is r operation is useful to containment tries the key containing the lowest address equal to the given address or CIDR pre finds the key containing the highest ac or equal to the given address or CIDR p `Higher` finds the key containing the lo greater than the given address or CIDR `Lower` finds the key containing the hig less than the given address or CIDR pr

Java	Go	Description
firstNode, firstAddedNode, lastNode, lastAddedNode, *firstEntry*, *lastEntry*, *firstKey*, *lastKey*, *pollFirstEntry*, *pollLastEntry*	FirstNode, FirstAddedNode, LastNode, LastAddedNode	Retrieves the node with the lowest or h key/address in the trie, according to th "AddedNode" variants give the same re cases, because the first and last node added node, the exceptions being whe are bigger than the non-added root, w are smaller than the non-added root, c are true and the trie is empty, in which added root may be returned.
elementContains, *elementContains*, elementsContaining, *elementsContaining*, longestPrefixMatch, longestPrefixMatchNode, *longestPrefixMatch*, *longestPrefixMatchEntry*, shortestPrefixMatch, shortestPrefixMatchNode	ElementContains, ElementsContaining, LongestPrefixMatch, LongestPrefixMatchNode, ShortestPrefixMatch, ShortestPrefixMatchNode	Retrieves or checks for the existence c trie whose subnet/address key contain individual address or CIDR prefix block distinction of these operations with the `contains`, which refers to containmer structure itself, rather than these meth subnet containment by a key/subnet/ad data structure. These operations refer of containment of addresses within sul variants of these operations that return results (containing, shortest prefix mat prefix match) return either all of the no whose elements/subnets are containin whose key is the largest-size subnet (s or the one node whose key is the smal (longest prefix)
elementsContainedBy, *elementsContainedBy*, removeElementsContainedBy	ElementsContainedBy, RemoveElementsContainedBy	Retrieves or removes the sub-trie (if ar are contained by the given individual a prefix block. If the given argument is a address, then the returned or removed only be a single node or null.
elementOverlaps, removeElementsIntersectedBy		Checks for the existence or removes th prefix block subnet or address keys ov subnet or address. When it comes to p prefix block overlapping or intersecting one contains the other, therefore these for nodes with subnet or address keys contained by the given address or sub

Java	Go	Description
size, *size*, nodeSize	Size, NodeSize	Operations returning the number of ele nodes) or the total number of nodes (b auto-generated) in the trie. The `size` constant time unless you are using a J `AddressTrieMap` or `AddressTrieSet` range, in which case it is linear time lik
getMatchingAddressCount		Returns the total number of individual contained by the subnet and address k added to the trie. This operation is use containment tries.
remap, remapIfAbsent	Remap, RemapIfAbsent	Allows for changes to trie mappings ba existing mappings, without having to d subsequent lookups, one to examine th second to change the trie based on the first lookup. Avoiding two lookups can beneficial with tries being accessed by threads, allowing for finer locking.
constructAddedNodesTree	ConstructAddedNodesTree	Produces a new associative trie derive The produced trie maps the root and th to their direct added sub-nodes in the Creates a non-binary tree structure fro nodes of the tree.
toAddedNodesTreeString	AddedNodesTreeString	Creates a more compact string represe (compared to the default string metho the root and the added nodes, one noc the non-binary tree relationships betw nodes visible. This operation visualizes produced by `constructAddedNodesTree` / `Construct`
toString, *toTrieString*, toTreeString	String, TreeString	Creates a string visualizing a trie or sub consists of a view of trie showing all th added nodes differentiated from non-a per line, and the binary tree relationshi nodes visible, with the element size of sub-trie visible.
compute, *computeIfAbsent*, *computeIfPresent*,		These map operations are specified by and implemented in `AddressTrieMap`

Java	Go	Description
getOrDefault, *merge*, *putIfAbsent*		convenience, avoiding multiple lookup Some of these use `remap` or `remapIf`
headMap, *tailMap*, *subMap*, *headSet*, *tailSet*, *subSet*		Operations creating a new `AddressTri` `AddressTrieSet` with a restricted rang the original `AddressTrieMap` or `Addre` the new set or map backed by the sam backed by the same trie, changes to th map will affect the original set or map trie, and vice-versa. After creating the you can use `asTrie` on that new set c you a new trie (and a new associated s allowing you to make changes without

Operations with Trie Nodes

The trie types allow direct access to the nodes, and provide many operations on the nodes directly. Many of the above operations on tries are also available on trie nodes. Many of those operations operate on the subtrie in which the given node is the root, such as the iterators, the containment operations, and others. Tries can also be altered by removing nodes directly.

Trie Iterators

Iterators are available in both Java and Go libraries.

Spliterators and streams are available in the Java library. In Java, in general you can obtain a corresponding stream for any spliterator by using `java.util.stream.StreamSupport.stream(Spliterator spliterator, boolean parallel)`, so if there exists a spliterator with no corresponding stream, that is how you can obtain a corresponding stream.

We separate these operations from the other trie operations because there is such a wide variety of methods for traversing tries or subtries.

Most of these traversal methods traverse the nodes, while some traverse the keys/addresses of the nodes, including those iterators and spliterators operating on the Java `AddressTrieSet` or `AddressTrieMap` types.

Of those that traverse the nodes, some traverse only the added nodes and elements, while others traverse all the nodes. The added nodes are those explicitly added to the trie, the non-added nodes are those auto-generated for the binary trie structure. Also note that non-added node can become added, and vice-versa.

Those operations that traverse the keys/addresses traverse only the added keys/addresses.

Java	Go	Description
iterator, descendingIterator, nodeIterator, allNodeIterator, spliterator, descendingSpliterator, stream, parallelStream, nodeSpliterator, allNodeSpliterator	Iterator, DescendingIterator, NodeIterator, AllNodeIterator	These traverse the trie in natural trie order, either in forward (ascending) or reverse (descending) order.
containingFirstIterator, containingFirstNodeIterator, containingFirstAllNodeIterator	ContainingFirstIterator, ContainingFirstAllNodeIterator	These traverse the trie with a pre-order binary tree traversal. Such a traversal visits parent (containing prefix blocks) nodes before sub-nodes. Sub-nodes can be visited either lower sub-node first (forwardSubNodeOrder) or upper sub-node first. The node iterators allow you to cache an object with either sub-node when visiting the parent node, an object that can be retrieved when visiting the respective sub-node.
containedFirstIterator, containedFirstNodeIterator, containedFirstAllNodeIterator	ContainedFirstIterator, ContainedFirstAllNodeIterator	These traverse the trie with a post-order binary tree traversal. Such a traversal visits parent (containing prefix blocks) nodes after sub-nodes. Sub-nodes can be visited either lower sub-node first (forwardSubNodeOrder) or upper sub-node first.
blockSizeIterator, blockSizeNodeIterator, blockSizeAllNodeIterator, blockSizeCachingAllNodeIterator	BlockSizeNodeIterator, BlockSizeAllNodeIterator, BlockSizeCachingAllNodeIterator	These traverse the trie from largest prefix blocks to smaller prefix blocks and finally to individual addresses. The node iterators allow you to cache

Java	Go	Description
		an object with either sub-node when visiting the parent node, an object that can be retrieved when visiting the respective sub-node

See the javadoc for TreeOps for more details on the orderings for the various traversals.

Operations with Collections

At this time, the two efficient optimized collections types are available only with the Java library.

The following operations are available with both collection options, `IPAddressSeqRange` and `IPAddressContainmentTrie` or through their shared interface, `IPAddressCollection`.

Java	Go	Description
add		Adds the individual addresses in the given address, subnet or sequential range to the collection
contains		Returns whether the collection contains all the individuals addresses in the given address, subnet, sequential range, or collection
overlaps		Returns whether the collection contains any of the individuals addresses in the given address, subnet or sequential range
remove		Removes the individual addresses in the given address, subnet or sequential range to the collection
getCount		Returns the number of individual addresses contained in the collection
equals		Returns whether this collections contains all the same individual addresses as the given collection
getLower		Returns the lowest valued individual address in the collection
getUpper		Returns the highest valued individual address in the collection
clear		Removes all addresses from the collection

Java	Go	Description
floor, ceiling, lower, higher		Finds the individual address in the collection is closest to the given individual address or CIDR prefix block. Ceiling finds the lowest individual address in the collection greater than or equal to the given address or CIDR prefix block. Floor finds the highest individual address in the collection less than or equal to the given address or CIDR prefix block. Higher finds the lowest individual address in the collection greater than the given address or CIDR prefix block. Lower finds the highest individual address in the collection less than the given address or CIDR prefix block.
isEmpty		Returns true if there are no addresses in the collection
isMultiple		Returns whether the collection contains multiple addresses
isSequential		Returns whether the range of addresses comprise a sequence of consecutive values. In other words, it is sequential if, given any two addresses in the collection, all addresses in-between are also in the collection. Empty collections are sequential. Collections with a single address are sequential.
includesZero		Returns whether the collection contains an address with the value of zero
includesMax		Returns whether the collection contains an address with the maximum value for addresses of the address version of the addresses in the collection
coverWithPrefixBlock		Returns the minimal-size prefix block that includes all addresses in the collection
coverWithSequentialRange		Returns the minimal-size sequential range that includes all addresses in the collection
iterator, spliterator, stream		Traverses through the individual addresses in the collection. Use `getCount` to get the traversed count.
clone		prodces a copy of the collection
toString		prodces a string visualizatio of the collection

Additional General Operations with Sequential Range List Collections

As a collection, sequential range lists have all the collection operations listed above, but they also have some additional operations that are not part of the `IPAddressCollection` interface.

Java	Go	Description
joinIntoList		This variant of the `add` methods joins the individual addresses in the given address, subnet or sequential range with the receiver list into a new list
removeIntoList		This variant of the `remove` methods removes the individual addresses in the given address, subnet or sequential range from the receiver list, putting the result into a new list
complementIntoList		Puts the complement of the list into a separate list
intersect		Changes the list to be the intersection with the given address, subnet or sequential range
complement		Changes the list to be the complement of itself
remove(addressIndex)		Removes from the collection the individual address at the given index in the sorted list
get(addressIndex)		Returns the individual address at the given index in the sorted list
increment		Returns the individual address at the given index inside or outside the sorted list
enumerate		Returns the index in the list of the given individual address
spanWithPrefixBlocks		Returns the minimal array of prefix block subnets that span all addresses contained in the list
spanWithSequentialBlocks		Returns the minimal array of sequential block subnets that span all addresses contained in the list
toCanonicalString		Creates a string showing the sequential ranges in the list, using the canonical string for the range boundaries of each contained sequential range
toNormalizedString		Creates a string showing the sequential ranges in the list, using the normalized string for the range boundaries of each contained sequential range

Additional Operations with Sequential Range Lists Accessing the Internal Range Structure

Sequential range lists do not allow direct access to the backing list of sequential ranges in a sequential range list, but they do allow indirect access through the following methods.

Java	Go	Description
getContainingSeqRange(addressIndex)		Returns the range containing the individual address at the given address index in the sorted sequential range list
getSegRange(rangeIndex)		Returns the range at the given sequential range index in the sorted list
removeSegRange(rangeIndex), removeSegRanges(fromIndex, toIndex)		Removes the range or ranges at the given sequential range index or indices in the sorted list
getSegRanges		returns an array of the sequential ranges in the list in order
seqRangeIterator		iterates through the sequential ranges in the list in order
getLowerSeqRange		returns the lowest sequential range in the ordered sequential range list
getUpperSeqRange		returns the highest sequential range in the ordered sequential range list
getSegRangeCount		returns the count of sequential ranges in the list
indexOfContainingSeqRange		If the list contains all individual addresses in the given address, subnet, sequential range, trie, or collection, returns the index of the lowest sequential range containing one of the addresses
indexOfContainingSeqRange		If the list overlaps some addresses in the given address, subnet, sequential range, or sequential range list, returns the index of the lowest sequential range containing one of the addresses

Parse String Representations of MAC Address

The library has support for working with MAC addresses.

Parsing is similar to IP address string parsing. `MACAddressString` is used to convert.

In Java, you can use one of `getAddress` or `toAddress`, the difference being whether parsing errors are handled by exception or not.

```
MACAddress address = new MACAddressString("01:02:03:04:0a:0b").getAddress();
if(address != null) {
  //use address
}
```

or

```
try {
  MACAddress address = new MACAddressString("01:02:03:04:0a:0b").toAddress();
  //use address
} catch (AddressStringException e) {
  String msg = e.getMessage(); // detailed message indicating issue
}
```

In Go, the equivalent code is similar, the difference between `GetAddress` and `ToAddress` being whether you want to return errors or not:

```
if address := ipaddr.NewMACAddressString("01:02:03:04:0a:0b").GetAddress(); address != nil {
  //use address
}
```

or

```
if address, err := ipaddr.NewMACAddressString("01:02:03:04:0a:0b").ToAddress(); err != nil {
  // handle error
} else {
  //use address
}
```

Various Formats of MAC Addresses

MAC Addresses are expected to be in hexadecimal. However, there is a number of accepted formats for MAC addresses:

```
aa:bb:cc:dd:ee:ff
aa-bb-cc-dd-ee-ff
aa bb cc dd ee ff
aabb.ccdd.eeff
aabbccddeeff
aabbcc-ddeeff
```

- `aa-bb-cc-dd-ee-ff` is the IEEE 802 format and common with Windows
- `aa:bb:cc:dd:ee:ff` is common in Unix-like systems and network tools
- `aabb.ccdd.eeff` is common with Cisco devices
- `aabbcc-ddeeff` is common with Hewlett Packard devices

For the non-segmented format (aabbccddeeff), all 12 digits are required to avoid ambiguity.

MAC addresses can be either 48 or 64 bits, and so for each 48-bit format there is a 64-bit equivalent:

```
aa:bb:cc:dd:ee:ff:11:22
aa-bb-cc-dd-ee-ff-11-22
aa bb cc dd ee ff 11 22
aabb.ccdd.eeff.1122
aabbccddeeff1122
aabbcc-ddeeff1122
```

For the non-segmented 64-bit format (aabbccddeeff1122), all 16 digits are required to avoid ambiguity.

As with IP addresses, you can specify ranges using '*' and '-' like aa-bb:*:*:cc:dd:ee. The range character for addresses that use the dash '-' character as a separator is '|', like aa|bb-*-*-cc-dd-ee.

Format Examples

For instance, the address 0a:0b:0c:0d:0e:0f can be represented many ways:

```java
static void parseMAC(String formats[]) {
  for(String format : formats) {
    System.out.println(new MACAddressString(format).getAddress());
  }
}

public static void main(String[] args) {
  String formats[] = {
```

```
    "a:b:c:d:e:f",
    "0a:0b:0c:0d:0e:0f",
    "a:b:c:d:e:f",
    "0a-0b-0c-0d-0e-0f",
    "0a0b0c-0d0e0f",
    "0a0b.0c0d.0e0f",
    "0a 0b 0c 0d 0e 0f",
    "0a0b0c0d0e0f"
  };
  parseMAC(formats);
}


func main() {
  formats := []string{
    "a:b:c:d:e:f",
    "0a:0b:0c:0d:0e:0f",
    "a:b:c:d:e:f",
    "0a-0b-0c-0d-0e-0f",
    "0a0b0c-0d0e0f",
    "0a0b.0c0d.0e0f",
    "0a 0b 0c 0d 0e 0f",
    "0a0b0c0d0e0f",
  }
  parseMAC(formats)
}

func parseMAC(formats []string) {
  for _, format := range formats {
    fmt.Println(ipaddr.NewMACAddressString(format).GetAddress())
  }
}
```

Output for either the Java or Go code:

```
0a:0b:0c:0d:0e:0f
0a:0b:0c:0d:0e:0f
0a:0b:0c:0d:0e:0f
0a:0b:0c:0d:0e:0f
0a:0b:0c:0d:0e:0f
0a:0b:0c:0d:0e:0f
0a:0b:0c:0d:0e:0f
0a:0b:0c:0d:0e:0f
```

Ranges can be specified in the same way as for IP addresses ('-' for a specific range or '*' for full range segments).

For instance, the address range ff:0f:aa-ff:00-ff:00-ff:00-ff can be represented many ways, in Java:

```java
static void parseMAC(String formats[]) {
  for(String format : formats) {
    System.out.println(new MACAddressString(format).getAddress());
  }
}

public static void main(String[] args) {
  String formats[] = {
    "ff:f:aa-ff:00-ff:00-ff:00-ff",
    "ff:f:aa-ff:*:*:*",
    "ff:0f:aa-ff:*:*:*",
    "ff-0f-aa|ff-*-*-*",
    "ff0faa|ff0fff-*",
    "ff0f.aa00-ffff.*",
    "ff 0f aa-ff * * *",
    "ff0faa000000-ff0fffffffff"
  };
  parseMAC(formats);
}
```

or in Go:

```go
func parseMAC(formats []string) {
  for _, format := range formats {
    fmt.Println(ipaddr.NewMACAddressString(format).GetAddress())
  }
}

func main() {
  formats := []string{
    "ff:f:aa-ff:00-ff:00-ff:00-ff",
    "ff:f:aa-ff:*:*:*",
    "ff:0f:aa-ff:*:*:*",
    "ff-0f-aa|ff-*-*-*",
    "ff0faa|ff0fff-*",
    "ff0f.aa00-ffff.*",
    "ff 0f aa-ff * * *",
    "ff0faa000000-ff0fffffffff",
  }
  parseMAC(formats)
}
```

Output for either the Java or Go code:

```
ff:0f:aa-ff:*:*:*
ff:0f:aa-ff:*:*:*
ff:0f:aa-ff:*:*:*
```

```
ff:0f:aa-ff:*:*:*
ff:0f:aa-ff:*:*:*
ff:0f:aa-ff:*:*:*
ff:0f:aa-ff:*:*:*
ff:0f:aa-ff:*:*:*
```

Delimited Segments

The range formats allow you to specify ranges of values. If you wish to parse addresses with delimited segment strings, methods are provided to do that.

In Java, if you wish to parse addresses in which segment values are delimited, then you can use the methods `parseDelimitedSegments(String)` and `countDelimitedAddresses(String)` in `MACAddressString`. `parseDelimitedSegments` will provide an iterator to traverse through the individual addresses.

In Go, this functionality is shared between IPAddress and MAC address parsing. We have already described the type `DelimitedAddressString` for this purpose.

For example, given "1,2:3:4,5:6:7:8", `countDelimitedAddresses` / `CountDelimitedAddresses` will return 4 for the possible combinations: "1:3:4:6:7:8", "1:3:5:6:7:8", "2:3:4:6:7:8" and "2:3:5:6:7:8". With each such string obtained from the iterator returned by `parseDelimitedSegments` / `ParseDelimitedSegments`, you can construct an instance of `MACAddressString`.

MAC Address Validation Options

Validation options allow you to restrict the allowed formats of MAC address strings.

The options are specified by the type `MACAddressStringParameters` which can be passed into `MACAddressString`.

You can restrict validation options using the appropriate builder types. Here is an example in Java:

```
MACAddressStringParameters MAC_ADDRESS_OPTIONS_EXAMPLE =
  new MACAddressStringParameters.Builder().
  allowEmpty(false).
  allowAll(false).
  getFormatBuilder().
    setRangeOptions(RangeParameters.*NO\_RANGE*).
    allowLeadingZeros(true).
    allowUnlimitedLeadingZeros(false).
    allowWildcardedSeparator(false).
    allowShortSegments(true).
```

```
    getParentBuilder().
  toParams();
```

and the equivalent code in Go:

```
var macAddressOptionsExample addrstrparam.MACAddressStringParams = new(addrstrparam.MACAddress
  AllowEmpty(false).
  AllowAll(false).
  GetFormatParamsBuilder().
  SetRangeParams(addrstrparam.NoRange).
  AllowLeadingZeros(true).
  AllowUnlimitedLeadingZeros(false).
  AllowWildcardedSeparator(false).
  AllowShortSegments(true).
  GetParentBuilder().
  ToParams()
```

The default options used by the library are permissive and not restrictive.

MAC Address Prefix Lengths

A ‘prefix length’ in this library is defined generally as the bit-length of the foremost portion of the address that is not specific to an individual address but common amongst a group of addresses. For IP addresses, this is explicitly defined as part of the address using the ‘/’ character. For IP addresses, the prefix is potentially variable and depends on how subnets have been allocated.

MAC Addresses don’t have the exact same concept of prefix length as IP addresses. But concept of prefix can be applied in an implicit sense. For instance, a MAC address is typically divided into an OUI (Organizationally Unique Identifier) and ODI (Organizational Defined Identifier), so you might consider the OUI bits as the prefix. There are other ways of assigning MAC address blocks as well, such as IAB (individual address block), or MA-S/MA-M/MA-L (MAC Address block small, medium, and large), in which a certain number of higher bits are provided as an identifier to organizations from which they can create various extended identifiers using the lower bits. There is generally a pre-defined set of high bits that can be considered a prefix. This prefix is not variable and was typically assigned by the IEEE. However, there is no explicit way to represent a MAC address string with an associated prefix, as there is with CIDR IP addresses, so a prefix with MAC addresses is implicit rather than explicit.

Within this library, the prefix for a MAC address is defined as the largest number of high bits for which an address represents all addresses with the same set of higher bits.

For instance, the prefix length of aa:*:*:*:*:* is 8 because the address represents all addresses that start with the same 8 bits “aa”. The prefix length of aa:*:cc:*:*:* is 24 because the address represents all

addresses that start with the same 16 bits aa:*:cc. The address aa:bb:cc:dd:ee:ff does not have a prefix or prefix length as it represents just a single address.

Once a MAC address as an associated prefix length, that prefix length remains the same while any operations are applied to the address (with the exception of operations that explicitly change the prefix, like `setPrefixLength`).

In summary, on the MAC side, the prefix length is implicit and based upon the address itself, while on the IP address side, the prefix length is explicitly defined.

MAC Address Operations

Many of the same address operations available for IP addresses are available for MAC addresses, including the prefix operations, the section and segment access methods, iterators, containment, and reversal of bits, bytes and segments. Because of their similarity, we will not relist the MAC equivalents.

The bit-reversal operations may be useful for for “MSB format”, “IBM format”, “Token-Ring format”, and “non-canonical form”, where the bits are reversed in each byte of a MAC address.

IPv6 – MAC Address Integration

There is a standardized procedure for converting MAC addresses to IPv6 given an IPv6 64-bit prefix, as described in IETF RFC 2464, RFC 3513, and RFC 4944.

It details how to combine a 48 bit MAC address with a 64-bit IPv6 network prefix to produce an associated 128-bit IPv6 address. This is done by first constructing a 64-bit extended unique IPv6 interface identifier (EUI-64) from the MAC address. This library has implemented the same MAC / IPv6 integration.

Construct IPv6 Address from MAC

Starting with a MAC address or section and with the IPv6 prefix, you can construct the associated IPv6 address with one of these constructors in Java:

```
public IPv6Address(IPv6Address prefix, MACAddress eui)

public IPv6Address(IPv6AddressSection section, MACAddress eui)

public IPv6Address(IPv6AddressSection section, MACAddressSection eui)

public IPv6Address(IPv6AddressSection section, MACAddressSection eui, CharSequence zone)
```

or one of these constructor functions in Go:

```
func NewIPv6AddressFromMAC(prefix *IPv6Address, suffix *MACAddress) (*IPv6Address, addrerr.Inc

func NewIPv6AddressFromMACSection(prefix *IPv6AddressSection, suffix *MACAddressSection) (*IPv

func NewIPv6AddressFromZonedMACSection(prefix *IPv6AddressSection, suffix *MACAddressSection,
```

Construct Link-local IPv6 Address from MAC

There are equivalent methods in the type `MACAddress` for producing the link local address which has a pre-defined prefix, or for producing the host (interface identifier) address section of an IPv6 address. In Java:

```
public IPv6Address toLinkLocalIPv6()

public IPv6AddressSection toEUI64IPv6()
```

In Go:

```
func (addr *MACAddress) ToLinkLocalIPv6() (*IPv6Address, addrerr.IncompatibleAddressError)

func (addr *MACAddress) ToEUI64IPv6() (*IPv6AddressSection, addrerr.IncompatibleAddressError)
```

EUI64 format

A MAC address is either 48 or 64 bits. To be converted to IPv6, the 48 bit address has segments inserted (two 1 byte MAC segments with value 0xfffe) to extend the address to 64 bits. For a 64 bit MAC address to be convertible, those same two segments must match the expected value of 0xffe. The following methods in MACAddress check will 64 bit addresses to ensure that are compatible with IPv6 by checking that the value of those two segments matches 0xfffe. Note that the asMAC argument allows you to extend using 0xffff rather than 0xfffe which is another manner by which a 48 bit MAC address can be extended to 64 bits. However, for purposes of extending to an IPv6 address, the `asMAC` argument should be false, so that the EUI-64 format is used.

In Java:

```
public boolean isEUI64(boolean asMAC)

public MACAddress toEUI64(boolean asMAC)
```

In Go:

```
func (addr *MACAddress) IsEUI64(asMAC bool) bool

func (addr *MACAddress) ToEUI64(asMAC bool) (*MACAddress, addrerr.IncompatibleAddressError)
```

The `MACAddressSection` and `IPv6AddressSection` have similar methods for constructing and identifying such sections. You can use those types of construct addresses in more customizable ways, whether using prepend or append operations, whether constructing segments directly.

Get MAC Address from IPv6 Address

To go the reverse direction IPv6 to MAC, there is a method in `IPv6Address` to produce a MAC address. In Java:

```
public MACAddress toEUI(boolean extended)
```

In Go:

```
func (addr *IPv6Address) ToEUI(extended bool) (*MACAddress, addrerr.IncompatibleAddressError)
```

In the Java library, there is another method in `IPv6AddressSection` that uses whatever part of the interface identifier is included in the section to produce a MAC address section:

```
public MACAddressSection toEUI(boolean extended)
```

For instance, if the IPV6 address section is the 8 bytes corresponding to the network prefix of an IPV6 address section, then the resulting MAC address section will be 0 bytes. If the IPV6 address section is the 8 bytes corresponding to the interface identifier, and that identifier has the required 0xfffe values in the 5th and 6th bytes, then the MAC address section will be the full 8 bytes of an EUI-64 MAC address.

Example Conversions from MAC Address to IPv6 Address

The following code is an example of constructing IPv6 addresses from a MAC address.

In Java:

```
public static void main(String args[]) {
  MACAddressString macStr = new MACAddressString("aa:bb:cc:dd:ee:ff");
  MACAddress macAddress = macStr.getAddress();
  IPv6Address linkLocal = macAddress.toLinkLocalIPv6();
```

```java
    System.out.println(linkLocal);

    IPAddressString ipv6Str = new IPAddressString("1111:2222:3333:4444::/64");
    IPv6Address ipv6Address = ipv6Str.getAddress().toIPv6();
    IPv6Address macIpv6 = new IPv6Address(ipv6Address, macAddress);
    System.out.println(macIpv6);
  }
```

In Go:

```go
  func main() {
    macStr := ipaddr.NewMACAddressString("aa:bb:cc:dd:ee:ff")
    var macAddress *ipaddr.MACAddress = macStr.GetAddress()
    var linkLocal *ipaddr.IPv6Address
    linkLocal, _ = macAddress.ToLinkLocalIPv6()
    fmt.Println(linkLocal)

    ipv6Str := ipaddr.NewIPAddressString("1111:2222:3333:4444::/64")
    var ipv6Address *ipaddr.IPv6Address = ipv6Str.GetAddress().ToIPv6()
    var macIpv6 *ipaddr.IPv6Address
    macIpv6, _ = ipaddr.NewIPv6AddressFromMAC(ipv6Address, macAddress)
    fmt.Println(macIpv6)
  }
```

Output from either the Java or Go code:

```
fe80::a8bb:ccff:fedd:eeff
1111:2222:3333:4444:a8bb:ccff:fedd:eeff/64
```

There is more sample code in a wiki example for Java and in a wiki example for Go.

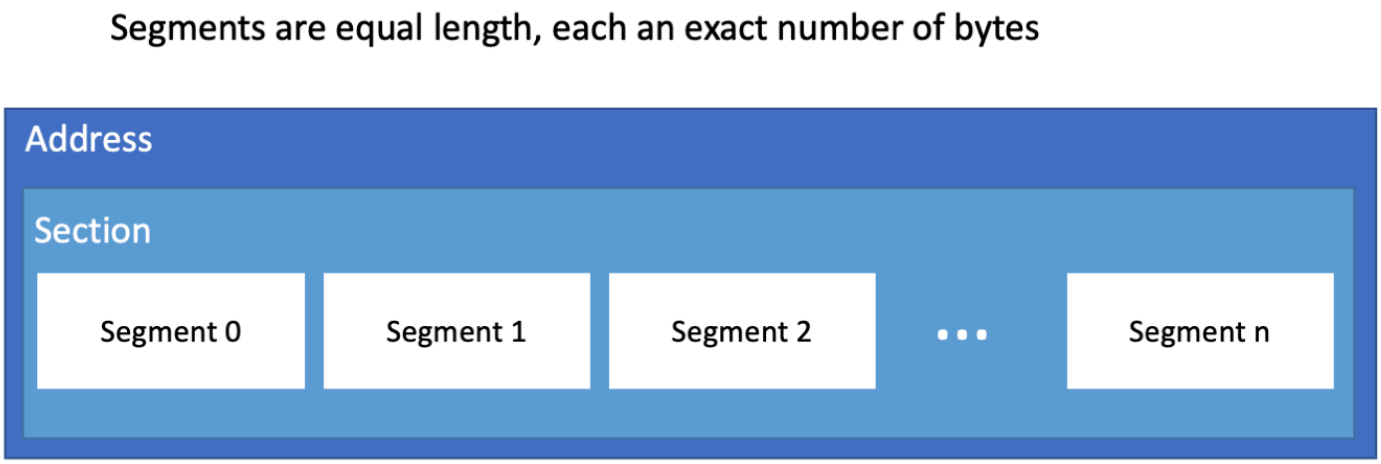
Address Framework

Much like there is a Java collections framework, there is an address framework to the IPAddress library. It is a unified set of inter-related interfaces, abstract implementations, and algorithms for all addresses and address components. It allows you to manipulate these items independently of implementation details, and provides standard interfaces to addresses and address components for code re-use and polymorphism. It represents the common structure of addresses, sections, segments, collections, aggregations, and the like.

You might wish to manipulate address components of different shapes and sizes transparently, or you may wish to manipulate different types of addresses or different address versions transparently. One element of the framework is the ability to convert any address component to bytes, whether division, segment, section, or address.

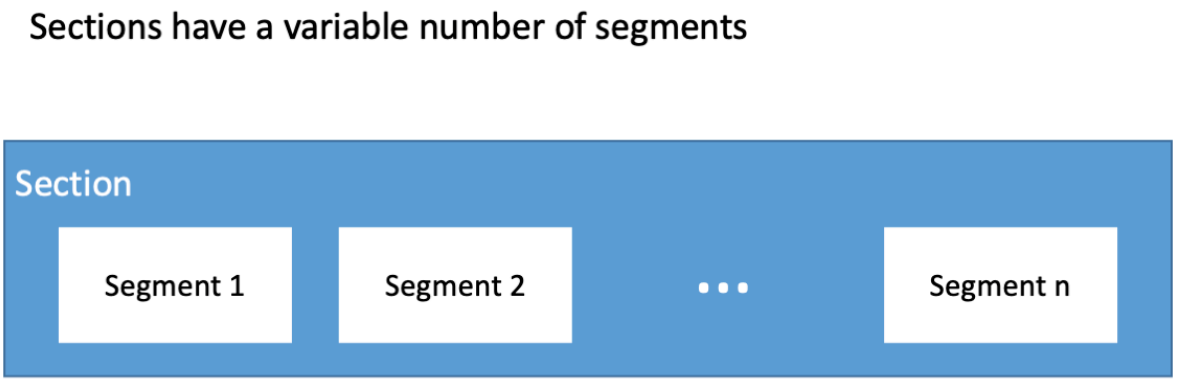
Address components

Addresses are composed of a single address section, which contains the full array of segments for a given address, which is 4 segments for IPv4, 8 for IPv6, and either 6 or 8 segments for MAC.



Examples
IPv4: 0.0.255.255
IPv6: fe80:0:0:0:204:61ff:aaab:bbbb
MAC: aa-bb-cc-dd-ee-ff

When an address is split into separate disparate sections, those sections can have a variable number of components. Sections can be manipulated individually and then reconstituted into a single section for a single address.



There is a more diversified hierarchy for non-standard address structures, in which addresses or address sections might be divided into divisions of unequal length, or divisions of non-integer byte-size.

Addresses can be split into sections or reconstituted into division groupings.
Divisions need not be equal length nor start and end on byte boundaries

Examples
IPv4: 0.0.65535
IPv6 with IPv4 embedded: fe80:0:0:0:204:61ff:254.157.241.86

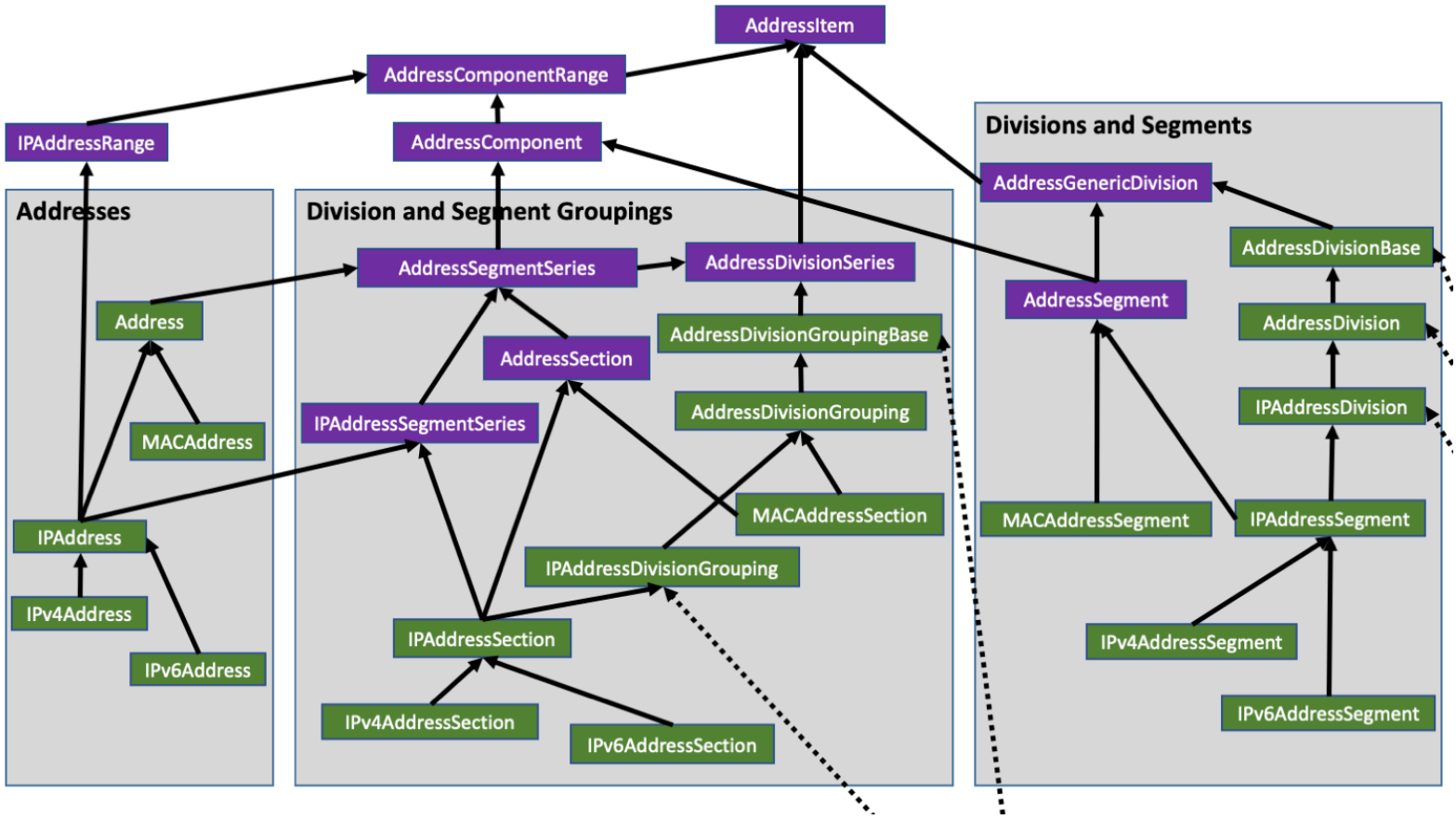
Address Component Framework

There exists a hierarchy for the standard and non-standard address and address component data structures. It includes types for the address components, which are addresses, sections of addresses, and segments of equal byte size inside those address sections. It includes a number of other types as well.

Java Address Component Framework

The address component hierarchy of interfaces (purple) and classes (green) in the Java library is shown in a diagram below. It is certainly not necessary to remember the hierarchy, it is simply useful for polymorphic code.

Most of the full class hierarchy for address structure showing addresses, sections, division groupings, segments and divisions is shown here, separated into the three primary categories described previously. The dashed lines indicate some less-prominent classes in the library not shown in the diagram. Also, a few small interfaces are omitted from the diagram.



Go Address Component Framework

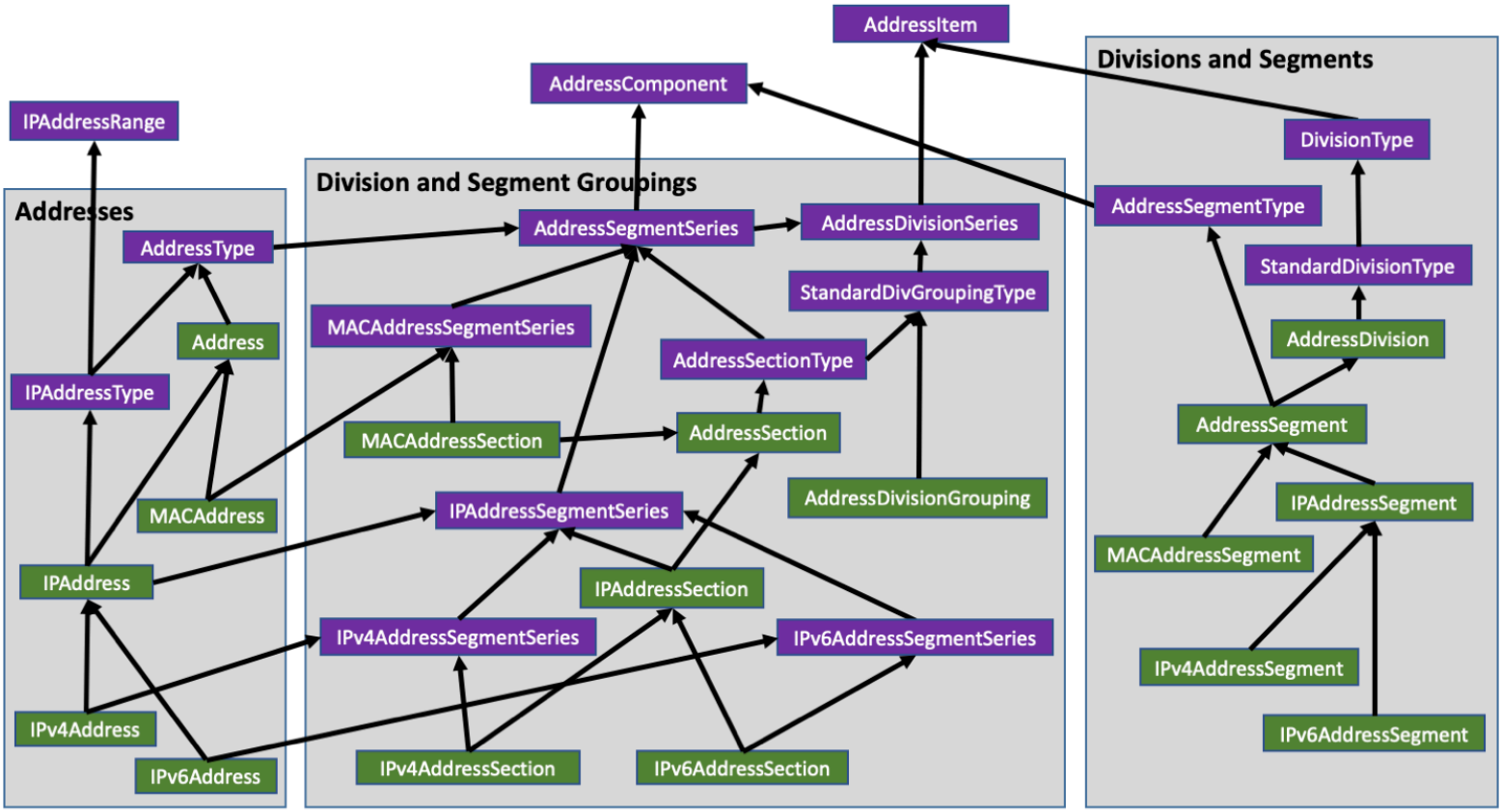
The Go library has an address component hierarchy similar in many ways while different in others.

Some differences derive from language differences, such as class inheritance in Java versus struct aggregation in Go. Other differences reflect the fact that an interface implementation is not declared explicitly in Go, and that there are some restrictions when embedding interfaces in Go.

Still, the Go library provides a similar framework that is useful for creating polymorphic code.

In lieu of inheritance, the Go library provides the types `AddressType`, `AddressSectionType`, `AddressSegmentType`, `StandardDivGroupingType`, `StandardDivisionType`, and `DivisionType` that allow for method parameters accepting arguments of similar address component structure. For instance, a method accepting IPv4, IPv6 and MAC addresses could use `AddressType` to represent any one of them.

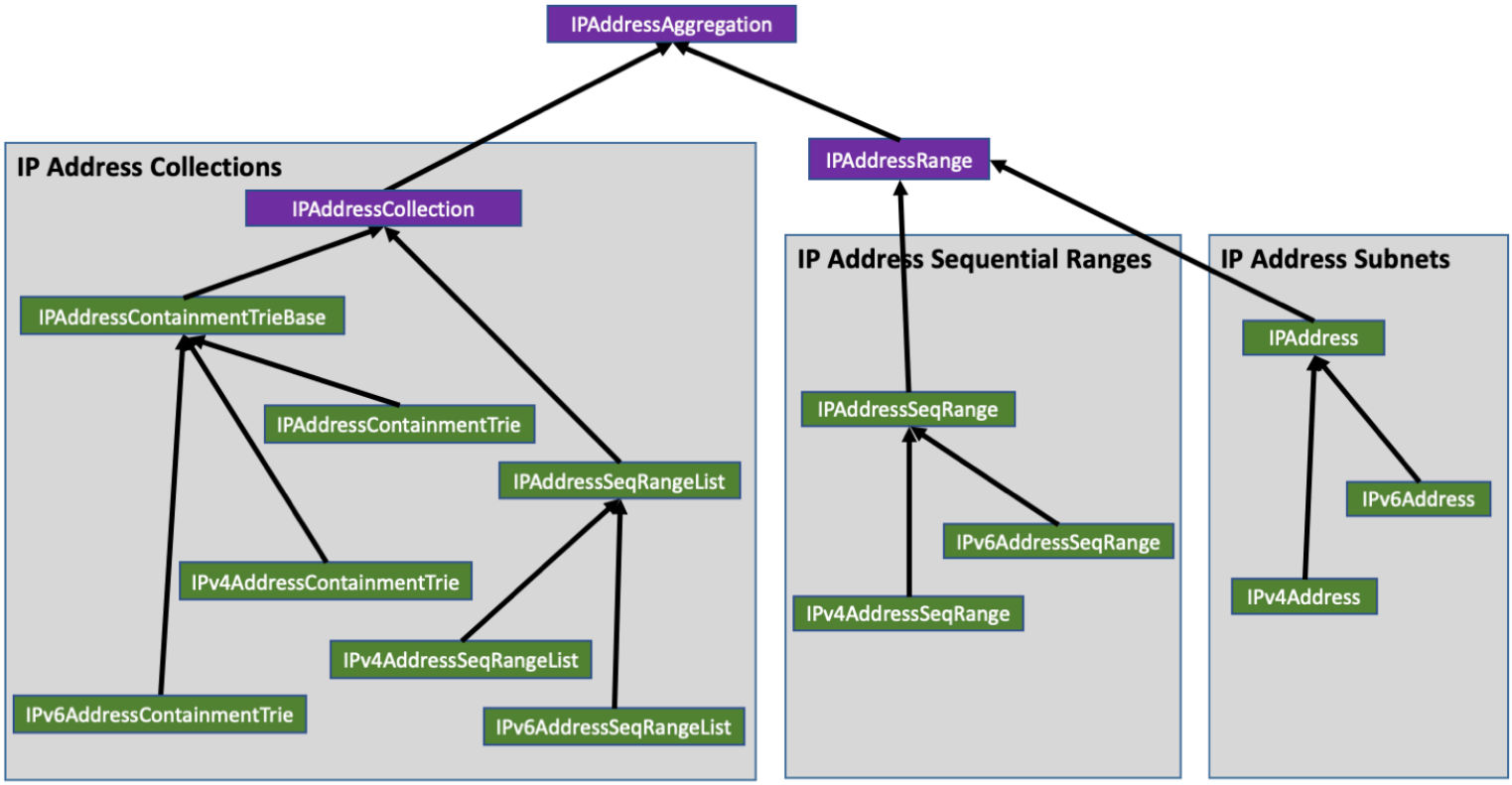
Meanwhile, just like in Java, other interfaces allow for polymorphism amongst types representing different component structures (one structure being an address, another being an address section, another being a segment, and so on). For instance, a method taking both address sections and addresses as arguments could use `AddressSegmentSeries` to do so. There are examples of such polymorphic code within the IPAddress library itself.



IP Address Aggregation Framework

The overall framework extends from addresses into subnets and the other possible aggregations of addresses that can be created, as shown in the following diagram.

You can represent individual IP addresses or subnets with `IPAddress`, or with IP-version-specific derivatives. You can represent a sequential range of IP addresses with `IPAddressSeqRange` and IP-version-specific derivatives. You can represent any collection of addresses with either an `IPAddressSeqRangeList` backed by a list of sequential ranges, or an `IPAddressContainmentTrie` backed by a trie of CIDR prefix blocks.



With a given address, subnet, or sequential range, you could choose to add all its contained individual addresses to a collection. Alternatively, the shared interface `IPAddressAggregation` allows you to store the collection alongside the address, subnet, or sequential range in some other data structure.

Conversion to String Representation of Address or Subnet

Most applications handle addresses as strings. The library has the `IPAddressString`, `HostName`, and `MACAddressString` types to parse strings into addresses. For the reverse, the library has a large number of methods to produce various strings with different formats from addresses.

Here is a list of string methods, in no specific order.

Note that, for a given address, the string produced by one of these methods might match those of other methods. They are not necessarily distinct from each other for all addresses.

All Address Types, HostName and IPAddressString

Java	Go	Description
toNormalizedString	ToNormalizedString	Produces a consistent and standardized string. For addresses, it is a string that is somewhat similar and consistent for all addresses the same type and version. Uses no segment compression in IPv6. Uses `xx:xx:xx:xx:xx:xx` for MAC address. Prints HostName instances in standard formats as expected by URLs and as dictated by RFCs, including port or service name and using square brackets for IPV6 addresses. Prints a standardized format for `IPAddressString` instances that cannot be converted to addresses (such as '*'). For `HostName` and `IPAddressString` instances constructed from invalid strings, prints the original string used to construct.
toString	String	For addresses, this produces the canonical string. For `HostName` and `IPAddressString` instances, this prints the original string that was used to construct the instance.

All Address Types

Java	Go	Description
toCanonicalString	ToCanonicalString	The canonical string is the string recommended by one or more RFCs or other common standard
toCompressedString	ToCompressedString	Produces the shortest representation of the address while remaining within the confines of the standard representation(s) of the address. For IPv6 this compresses all compressible segments
toHexString	ToHexString	non-segmented base 16

IP Addresses and HostName

Java	Go	Description
toNormalizedWildcardString	ToNormalizedWildcardString	similar to the normalized string, but uses wildcards and does not print prefix

Java	Go	Description
		length for addresses that have prefix lengths, will use wildcards to indicate ranges instead

IP Addresses Only

Java	Go	Description
toFullString	ToFullString	a string which maintains a common length, a string with no compressed segments and all segments of full length, which is 4 characters for IPv6 segments and 3 characters for IPv4 segments
toCanonicalWildcardString	ToCanonicalWildcardString	similar to the canonical string, but uses wildcards and does not print prefix length for addresses that have prefix lengths
toCompressedWildcardString	ToCompressedWildcardString	similar to the compressed string, but uses wildcards and does not print prefix length for addresses that have prefix lengths
toSQLWildcardString	ToSQLWildcardString	Similar to the normalized wildcard string, but uses the SQL wildcards `%` and '_'

Java	Go	Description
toPrefixLengthString	ToPrefixLenString	Produces a string with CIDR prefix length if it has a prefix length, and compresses the host for IPv6. For IPv4 it is the same as the canonical string
toSubnetString	ToSubnetString	Produces the normalized wildcard string for IPv4 and the prefix length string for IPv6
toReverseDNSLookupString	ToReverseDNSString	produces the reverse DNS lookup string
toOctalString	ToOctalString	non-segmented base 8, optionally with a ‘0’ prefix to indicate octal
toBinaryString	ToBinaryString	non-segmented base 2, all ones and zeros
toSegmentedBinaryString	ToSegmentedBinaryString	segmented base 2, all ones and zeros
toConvertedString		For IPv6, if the IPv6 address can be converted to IPv4, produces the IPv6 mixed string. For IPv4 this produces the canonical string.
toUNCHostName	ToUNCHostName	produces the Microsoft UNC path component
toNormalizedString(IPStringOptions)	ToNormalizedString(IPStringOptions)	use this method to produce your own customized string

IPv6 Only

Java	Go	Description
toMixedString	ToMixedString	Produces a string with the mixed IPv6/IPv4 string format like `a:b:c:d:e:f:255.0.255.255`
toBase85String	ToBase85String	Produces the Base 85 string, see RFC 1924, the Java wiki example, or the Go wiki example

MAC Address Only

Java	Go	Description
toColonDelimitedString	ToColonDelimitedString	This is the normalized string for MAC addresses, and has the format `xx:xx:xx:xx:xx:xx`
toDashedString	ToDashedString	This is the canonical string for MAC addresses, and has the format `xx-xx-xx-xx-xx-xx`
toDottedString	ToDottedString	This has the format `xxxx.xxxx.xxxx` , with the segments being 2 bytes each, rather than the other MAC strings in which segments are 1 byte each
toSpaceDelimitedString	ToSpaceDelimitedString	This has the format `xx xx xx xx xx xx`

More Details and Examples

`AddressSegmentSeries` implementations (such as `Address` and `AddressSection`) have methods that produce strings in various formats.

This Java code produces those assorted strings from a selection of six addresses and subnets:

```
public static void main(String[] args) {
  printStrings(new MACAddressString("a:bb:c:dd:e:ff").getAddress());
  printStrings(new MACAddressString("a:bb:c:*").getAddress());
  printStrings(new IPAddressString("a:bb:c::/64").getAddress());
  printStrings(new IPAddressString("1.2.3.4").getAddress());
  printStrings(new IPAddressString("1.2.0.0/16").getAddress());
  printStrings(new IPAddressString("a:bb:c::dd:e:ff").getAddress());
}
```

```java
static void printStrings(AddressSegmentSeries series) {
  System.out.println(series.toCanonicalString());
  System.out.println(series.toNormalizedString());
  System.out.println(series.toCompressedString());
  System.out.println(series.toHexString(true));
  System.out.print("lower: " + series.getLower() + " bytes:");
  System.out.println(Arrays.toString(unsigned(series.getBytes())));
  System.out.print("upper: " + series.getUpper() + " bytes:");
  System.out.println(Arrays.toString(unsigned(series.getUpperBytes())));
  System.out.println();
}

static int[] unsigned(byte bytes[]) {
  int[] result = new int[bytes.length];
  for (int i = 0; i < result.length; i++) {
    result[i] = Byte.toUnsignedInt(bytes[i]);
  }
  return result;
}
```

This is the equivalent Go code:

```go
func main() {
        printStrings(ipaddr.NewMACAddressString("a:bb:c:dd:e:ff").GetAddress())
        printStrings(ipaddr.NewMACAddressString("a:bb:c:*").GetAddress())
        printStrings(ipaddr.NewIPAddressString("a:bb:c::/64").GetAddress())
        printStrings(ipaddr.NewIPAddressString("1.2.3.4").GetAddress())
        printStrings(ipaddr.NewIPAddressString("1.2.0.0/16").GetAddress())
        printStrings(ipaddr.NewIPAddressString("a:bb:c::dd:e:ff").GetAddress())
}

func printStrings(series ipaddr.AddressType) {
        fmt.Println(series.ToCanonicalString())
        fmt.Println(series.ToNormalizedString())
        fmt.Println(series.ToCompressedString())
        hexStr, _ := series.ToHexString(true)
        fmt.Println(hexStr)
        fmt.Print("lower: " + series.ToAddressBase().GetLower().String() + " bytes:")
        fmt.Println(series.Bytes())
        fmt.Print("upper: " + series.ToAddressBase().GetUpper().String() + " bytes:")
        fmt.Println(series.UpperBytes())
        fmt.Println()
}
```

Output from both the Java and Go code:

```
0a-bb-0c-dd-0e-ff
0a:bb:0c:dd:0e:ff
a:bb:c:dd:e:ff
```

```
0x0abb0cdd0eff
lower: 0a:bb:0c:dd:0e:ff bytes:[10, 187, 12, 221, 14, 255]
upper: 0a:bb:0c:dd:0e:ff bytes:[10, 187, 12, 221, 14, 255]

0a-bb-0c-*-*-*
0a:bb:0c:*:*:*
a:bb:c:*:*:*
0x0abb0c000000-0x0abb0cffffff
lower: 0a:bb:0c:00:00:00 bytes:[10, 187, 12, 0, 0, 0]
upper: 0a:bb:0c:ff:ff:ff bytes:[10, 187, 12, 255, 255, 255]

a:bb:c::/64
a:bb:c:0:0:0:0:0/64
a:bb:c::/64
0x000a00bb000c00000000000000000000-0x000a00bb000c0000ffffffffffffffff
lower: a:bb:c::/64 bytes:[0, 10, 0, 187, 0, 12, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
upper: a:bb:c:0:ffff:ffff:ffff:ffff/64 bytes:[0, 10, 0, 187, 0, 12, 0, 0, 255, 255, 255, 255,

1.2.3.4
1.2.3.4
1.2.3.4
0x01020304
lower: 1.2.3.4 bytes:[1, 2, 3, 4]
upper: 1.2.3.4 bytes:[1, 2, 3, 4]

1.2.0.0/16
1.2.0.0/16
1.2.0.0/16
0x01020000-0x0102ffff
lower: 1.2.0.0/16 bytes:[1, 2, 0, 0]
upper: 1.2.255.255/16 bytes:[1, 2, 255, 255]

a:bb:c::dd:e:ff
a:bb:c:0:0:dd:e:ff
a:bb:c::dd:e:ff
0x000a00bb000c0000000000dd000e00ff
lower: a:bb:c::dd:e:ff bytes:[0, 10, 0, 187, 0, 12, 0, 0, 0, 0, 0, 221, 0, 14, 0, 255]
```

The `IPAddress` and `IPAddressSection` types and their associated version-specific types have additional methods to produce specific strings representing the address.

Here is example Java code:

```
public static void main(String[] args) {
  IPAddress address = new IPAddressString("a:b:c::e:f").getAddress();
  print(address);
  address = new IPAddressString("a:b:c::").getAddress();
  print(address);
```

```
    //a:b:c:\*:: cannot be represented as a range of two single values
    //so this throws exception in toBase85String(), toBinaryString(), and toHexString()
    address = new IPAddressString("a:b:c:*::").getAddress();
    print(address);
  }

  static void print(IPAddress address) {
    System.out.println(address.toCanonicalString());
    System.out.println(address.toFullString());
    System.out.println(address.toNormalizedString());
    System.out.println(address.toSQLWildcardString());
    System.out.println(address.toSubnetString());
    try {
      if(address.isIPv6()) {
        System.out.println(address.toIPv6().toMixedString());
        System.out.println(address.toIPv6().toBase85String());
      }
      System.out.println(address.toBinaryString());
      System.out.println(address.toHexString(true));
    } catch(IncompatibleAddressException e) {}
    System.out.println();
  }
```

Here is equivalent Go code:

```
  func main() {
          address := ipaddr.NewIPAddressString("a:b:c::e:f").GetAddress()
          print(address)
          address = ipaddr.NewIPAddressString("a:b:c::").GetAddress()
          print(address)

          //a:b:c:\*:: cannot be represented as a range of two single values
          //so an error and empty string is returned from ToBase85String(), ToBinaryString(), an
          address = ipaddr.NewIPAddressString("a:b:c:*::").GetAddress()
          print(address)
  }

  func print(address *ipaddr.IPAddress) {
          fmt.Println(address.ToCanonicalString())
          fmt.Println(address.ToFullString())
          fmt.Println(address.ToNormalizedString())
          fmt.Println(address.ToSQLWildcardString())
          fmt.Println(address.ToSubnetString())
          if address.IsIPv6() {
                  str, _ := address.ToIPv6().ToMixedString()
                  fmt.Println(str)
                  str, _ = address.ToIPv6().ToBase85String()
                  fmt.Println(str)
          }
          str, _ := address.ToBinaryString(false)
          fmt.Println(str)
```

```
        str, _ = address.ToHexString(true)
        fmt.Println(str)
        fmt.Println()
}
```

Output from both the Java and Go code:

```
a:b:c::e:f
000a:000b:000c:0000:0000:0000:000e:000f
a:b:c:0:0:0:e:f
a:b:c:0:0:0:e:f
a:b:c::e:f
a:b:c::0.14.0.15
00|N0s0$N0-%*(tF74+!
0000000000001010000000000000101100000000000011000000000000000000000000000000000000000000000000
0x000a000b000c000000000000000e000f

a:b:c::
000a:000b:000c:0000:0000:0000:0000:0000
a:b:c:0:0:0:0:0
a:b:c:0:0:0:0:0
a:b:c::
a:b:c::
00|N0s0$N0-%*(tF5l-X
0000000000001010000000000000101100000000000011000000000000000000000000000000000000000000000000
0x000a000b000c0000000000000000000000

a:b:c:*::
000a:000b:000c:0000-ffff:0000:0000:0000:0000
a:b:c:*:0:0:0:0
a:b:c:%:0:0:0:0
a:b:c:*::
a:b:c:*::
```

UNC Strings

There is a method to produce the UNC IP-literal string.

In Java:

```
IPAddressString ipAddressString = new IPAddressString("2001:db8::1");
IPAddress address = ipAddressString.getAddress();
System.out.println(address.toUNCHostName());

ipAddressString = new IPAddressString("1.2.3.4");
```

```
address = ipAddressString.getAddress();
System.out.println(address.toUNCHostName());
```

The Go code for the same:

```
var ipAddressString *ipaddr.IPAddressString = ipaddr.NewIPAddressString("2001:db8::1")
var address *ipaddr.IPAddress = ipAddressString.GetAddress()
fmt.Println(address.ToUNCHostName())

ipAddressString = ipaddr.NewIPAddressString("1.2.3.4")
address = ipAddressString.GetAddress()
fmt.Println(address.ToUNCHostName())
```

Output from both the Java and Go code:

```
2001-db8-0-0-0-0-0-1.ipv6-literal.net
1.2.3.4
```

DNS Lookup Strings

There is a method to produce the string that is used for reverse DNS lookup. If you wish to do a reverse DNS lookup for a subnet rather than a full address, you can use `getNetworkSection()` / `GetNetworkSection` to provide the network section you wish to lookup. You can specify the prefix length in either the string itself or the call to get the network section.

In Java:

```
IPAddressString ipAddressString = new IPAddressString("2001:db8::1");
IPAddress address = ipAddressString.getAddress();
System.out.println(address.toReverseDNSLookupString());
IPAddressSection addressSection = address.getNetworkSection(64);
System.out.println(addressSection.toReverseDNSLookupString());

//same with prefix
ipAddressString = new IPAddressString("2001:db8::1/64");
address = ipAddressString.getAddress();
System.out.println(address.toReverseDNSLookupString());
addressSection = address.getNetworkSection();
System.out.println(addressSection.toReverseDNSLookupString());

//prefix block
ipAddressString = new IPAddressString("2001:db8::/64");
address = ipAddressString.getAddress();
System.out.println(address.toReverseDNSLookupString());
```

```java
addressSection = address.getNetworkSection();
System.out.println(addressSection.toReverseDNSLookupString());
```

and in Go:

```go
var ipAddressString *ipaddr.IPAddressString = ipaddr.NewIPAddressString("2001:db8::1")
var address *ipaddr.IPAddress = ipAddressString.GetAddress()
str, _ := address.ToReverseDNSString()
fmt.Println(str)
addressSection := address.GetNetworkSectionLen(64)
str, _ = addressSection.ToReverseDNSString()
fmt.Println(str)

// same with prefix
ipAddressString = ipaddr.NewIPAddressString("2001:db8::1/64")
address = ipAddressString.GetAddress()
str, _ = address.ToReverseDNSString()
fmt.Println(str)
addressSection = address.GetNetworkSection()
str, _ = addressSection.ToReverseDNSString()
fmt.Println(str)

// prefix block
ipAddressString = ipaddr.NewIPAddressString("2001:db8::/64")
address = ipAddressString.GetAddress()
str, _ = address.ToReverseDNSString()
fmt.Println(str)
addressSection = address.GetNetworkSection()
str, _ = addressSection.ToReverseDNSString()
fmt.Println(str)
```

Output from both the Java and Go code:

```
1.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.8.b.d.0.1.0.0.2.ip6.arpa
0.0.0.0.0.0.0.0.8.b.d.0.1.0.0.2.ip6.arpa
1.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.8.b.d.0.1.0.0.2.ip6.arpa
0.0.0.0.0.0.0.0.8.b.d.0.1.0.0.2.ip6.arpa
*.*.*.*.*.*.*.*.*.*.*.*.*.*.*.*.0.0.0.0.0.0.0.0.8.b.d.0.1.0.0.2.ip6.arpa
0.0.0.0.0.0.0.0.8.b.d.0.1.0.0.2.ip6.arpa
```

General String Methods

The canonical string and compressed string are available from any address or address section.

The normalized string and hexadecimal string are available from any address component, including segments and sections.

Prefix Length Indicator in Strings

Typically prefix lengths will be added to IP strings. To choose to print wildcards `*` and range characters `-` as opposed to using prefix length, there are additional methods.

Here is sample code in Java:

```
public static void main(String[] args) {
  IPAddress address = new IPAddressString("a:b:c::/64").getAddress();
  print(address);
  address = new IPAddressString("a:b:c:*::/64").getAddress();
  print(address);
}

static void print(IPAddress address) {
  System.out.println(address.toCanonicalString());
  System.out.println(address.toCanonicalWildcardString());
  System.out.println(address.toFullString());
  System.out.println(address.toNormalizedString());
  System.out.println(address.toSQLWildcardString());
  System.out.println();
}
```

and the equivalent code in Go:

```
func main() {
        address := ipaddr.NewIPAddressString("a:b:c::/64").GetAddress()
        print(address)
        address = ipaddr.NewIPAddressString("a:b:c:*::/64").GetAddress()
        print(address)
}

func print(address *ipaddr.IPAddress) {
        fmt.Println(address.ToCanonicalString())
        fmt.Println(address.ToCanonicalWildcardString())
        fmt.Println(address.ToFullString())
        fmt.Println(address.ToNormalizedString())
        fmt.Println(address.ToSQLWildcardString())
        fmt.Println()
}
```

Output from both the Java and Go code:

```
a:b:c::/64
a:b:c:0:*:*:*:*
```

```
000a:000b:000c:0000:0000:0000:0000:0000/64
a:b:c:0:0:0:0:0/64
a:b:c:0:%:%:%:%

a:b:c:*::/64
a:b:c:*:*:*:*:*
000a:000b:000c:0000-ffff:0000:0000:0000:0000/64
a:b:c:*:0:0:0:0/64
a:b:c:%:%:%:%:%
```

Strings Dependent on IP Version

Some strings are version-dependent.

Here we show the subnet string and the IPv4-mapped converted string in Java:

```java
// toSubnetString() prefers '*' for IPv4 and prefix for IPv6
System.out.println(new IPAddressString("1.2.0.0/16").getAddress().toSubnetString());
System.out.println(new IPAddressString("a:b::/64").getAddress().toSubnetString());

// converts IPv4-mapped to a:b:c:d:e:f:1.2.3.4 notation
System.out.println(new IPAddressString("::ffff:a:b").getAddress().toConvertedString());
```

The Go library does not have an `IPAddress` method for the IPv4-mapped converted string since implicit conversion was not added to the Go library, in the spirit of doing such things explicitly in the Go language. However, the code here is equivalent. and in Go:

```go
// ToSubnetString prefers '*' for IPv4 and prefix for IPv6
fmt.Println(ipaddr.NewIPAddressString("1.2.0.0/16").GetAddress().ToSubnetString())
fmt.Println(ipaddr.NewIPAddressString("a:b::/64").GetAddress().ToSubnetString())

// converts IPv4-mapped to a:b:c:d:e:f:1.2.3.4 notation
fmt.Println(convertedString(ipaddr.NewIPAddressString("::ffff:a:b").GetAddress()))

func convertedString(address *ipaddr.IPAddress) string {
	if address.IsIPv6() && address.ToIPv6().IsIPv4Mapped() {
		str, _ := address.ToIPv6().ToMixedString()
		return str
	}
	return address.ToNormalizedString()
}
```

Output from both the Java and Go code:

```
1.2.*.*
a:b::/64
::ffff:0.10.0.11
```

Collections of IP Address Strings

With the Java library, you can produce collections of strings:

```java
public static void main(String[] args) {
  IPAddress address = new IPAddressString("a:b:c::e:f").getAddress();
  print(address);
}

private static void print(IPAddress address) {
  //print(address.toAllStrings()); produces many strings
  print(address.toStandardStrings());
}

public static void print(String strings[]) {
  for(String str : strings) {
    System.out.println(str);
  }
}
```

Output:

```
a:b:c:0:0:0:0.14.0.15
a:b:c:0:0:0:000.014.000.015
000a:000b:000c:0000:0000:0000:0.14.0.15
000a:000b:000c:0000:0000:0000:000.014.000.015
A:B:C:0:0:0:0.14.0.15
A:B:C:0:0:0:000.014.000.015
000A:000B:000C:0000:0000:0000:0.14.0.15
000A:000B:000C:0000:0000:0000:000.014.000.015
a:b:c::0.14.0.15
a:b:c::000.014.000.015
000a:000b:000c::0.14.0.15
000a:000b:000c::000.014.000.015
A:B:C::0.14.0.15
A:B:C::000.014.000.015
000A:000B:000C::0.14.0.15
000A:000B:000C::000.014.000.015
a:b:c:0:0:0:e:f
000a:000b:000c:0000:0000:0000:000e:000f
A:B:C:0:0:0:E:F
```

```
000A:000B:000C:0000:0000:0000:000E:000F
a:b:c::e:f
000a:000b:000c::000e:000f
A:B:C::E:F
000A:000B:000C::000E:000F
```

The String collections can be customized with `toNormalizedString(StringOptions params)`

Note that string collections never have duplicate strings. The String collections can be customized with `toStrings(IPStringBuilderOptions options)` .

Containment and Membership

To check whether an IP address is contained by a subnet, in Java:

```java
IPAddress address = new IPAddressString("1.2.0.0/16").getAddress();
System.out.println(address.contains(new IPAddressString("1.2.3.4").getAddress()));
System.out.println(address.contains(new IPAddressString("1.2.3.0/24").getAddress()));
System.out.println(address.contains(new IPAddressString("1.2.3.0/25").getAddress()));
System.out.println(address.contains(new IPAddressString("1.1.0.0").getAddress()));
```

and the equivalent Go code:

```go
var address *IPAddress = ipaddr.NewIPAddressString("1.2.0.0/16").GetAddress()
fmt.Println(address.Contains(ipaddr.NewIPAddressString("1.2.3.4").GetAddress()))
fmt.Println(address.Contains(ipaddr.NewIPAddressString("1.2.3.0/24").GetAddress()))
fmt.Println(address.Contains(ipaddr.NewIPAddressString("1.2.3.0/25").GetAddress()))
fmt.Println(address.Contains(ipaddr.NewIPAddressString("1.1.0.0").GetAddress()))
```

Output:

```
true
true
true
false
```

The `contains` method is not restricted to IP addresses or IP address prefixed addresses. There is a `contains` method for every `Address` or `AddressSection` .

`IPAddressString` has a `contains` methods as well, and also some additional containment methods, `prefixContains` and `prefixEquals` . These `IPAddressString` containment methods can have superior

performance when checking containment when starting from strings, because in many cases containment can be determined by looking at the strings alone, avoiding address object creation and numeric comparisons.

For checking containment of an address or subnet in a large number of subnets, or to check containment of a large number of addresses or subnets in a subnet, use the address trie data structure, or an IP address collection which can be either an IP address sequential range list, or an IP address containment trie. When using an address trie, it is the `elementContains` method that tests whether an element of the trie contains the given subnet or address. The `contains` method tests for an exact match.

The IPAddressAggregation interface has `contains` and `overlaps` methods that check for containment of individual addresses, subnets, or sequential ranges. So that means you can use those methods to test for containment of any subnet, address range, or collection.

The wiki has a wide variety of examples showing how to test for containment or make comparisons with a wide variety of address and collection types.

When it comes to accessing individual members, There is also an assortment of iterators for addresses, sections, and segments which can access individual members. There is an `iterator` / `Iterator` , `getLower` / `GetLower` method and `getUpper` / `GetUpper` method for every address component. You can use the `increment` / `Increment` or `get` / `Get` methods to access individual members or sequential ranges or range lists. The `increment` / `Increment` method is also available with any subnet or subnet section.

Sorting and Comparisons

Comparing and sorting can be useful for storing addresses in certain types of data structures. With the Java library, all of the core classes implement `java.lang.Comparable` . In fact, any `AddressItem` is comparable to any other, which covers almost every type in the address framework. Different representations of the same address or subnet are considered equal. Different representations of the same set of addresses are considered equal. However, `HostName` instances are not equal to `IPAddressString` instances nor `IPAddress` instances, and instances of `IPAddressString` are not equal to instances of `IPAddress` , not even when representing the same address or subnet.

The library provides the type `AddressComparator` and some comparator implementations for comparison purposes. Address classes use the type `CountComparator` for their natural ordering. When comparing subnets, you can either emphasize the count of addresses, or you can emphasize the values of the lower or upper address represented by the subnet, and comparators are provided for those variations.

The address tries are sorted and provide their own comparator, which matches the comparators above with respect to individual addresses. It differs to some degree when comparing subnets, which must be prefix blocks to use with the trie comparator. When comparing a prefix block to a second address or block with

larger prefix, and the first prefix matches the same bits in the second, the trie comparator orders by the bit that follows the first prefix in the second. For example, if a block with prefix length 12 is compared to a prefix block of larger prefix length, then the comparison first examines the common prefix bits, the first 12 bits, to determine order. If those 12 bits are the same, then the comparison examines bit 13 in the other block with the longer prefix. A bit of value one indicates the other block is greater than the first, a bit of value zero indicates the other block is less than the first.

The two collection implementations, `IPAddressContainmentTrie` and `IPAddressSeqRange` are both sorted, which makes a number of operations much more efficient by enabling binary search.

DNS Resolution and URLs

If you have a string that can be a host or an address and you wish to resolve to an address, create a `HostName` and use the `toAddress` / `ToAddress` method. If you wish to obtain a string representation to be part of a URL, use the normalized string from `toNormalizedString` / `ToNormalizedString` .

Make your IPv4 App work with IPv6

If you want to make your app work with IPv6, that is a wise decision. IPv6 penetration is slow but steady, past 50% in some countries.

But you have no IPv6 testing and a lot of code.

Start by replacing your IPv4 code with the types and operations in this library. But don't use the IPv4-specific types like IPv4Address everywhere, use `IPAddressString` , `IPAddress` , and the other polymorphic types, along with their polymorphic operations such as `toString` , `contains` , and `iterator` that avoid exposing the specifics of IPv4. If your app uses network or host masks, start working with CIDR prefix lengths instead. Use methods like `getBlockMaskPrefixLength` to store prefix lengths, even if the app continues to use masking.

When you're ready, use your existing regression tests, ensuring the app still runs as always on IPv4. Once your existing IPv4 tests all pass, you are already most of the way to supporting IPv6, without having written a single line of IPv6 code. You may even be ready to try it out with IPv6.

This site is open source. Improve this page.